



**UNIVERSITÀ
DI PARMA**

DIPARTIMENTO DI SCIENZE MATEMATICHE, FISICHE E INFORMATICHE

Corso di Laurea Triennale in Informatica

Un workflow LLM per la creazione interattiva di basi di dati: modulo converter

Candidato

Fabio Meloni

Relatore

Prof. Vincenzo Bonnici

Quali problemi presenta il salvare i dati in un foglio di calcolo?

Quali problemi presenta il salvare i dati in un foglio di calcolo?

1) Difficile garantire integrità referenziale dei dati;

Quali problemi presenta il salvare i dati in un foglio di calcolo?

- 1) Difficile garantire integrità referenziale dei dati;
- 2) Nessun audit trail per registrare le modifiche;

Quali problemi presenta il salvare i dati in un foglio di calcolo?

- 1) Difficile garantire integrità referenziale dei dati;
- 2) Nessun audit trail per registrare le modifiche;
- 3) Scarsa scalabilità in caso di dataset di grandi dimensioni;

Quali problemi presenta il salvare i dati in un foglio di calcolo?

- 1) Difficile garantire integrità referenziale dei dati;
- 2) Nessun audit trail per registrare le modifiche;
- 3) Scarsa scalabilità in caso di dataset di grandi dimensioni;
- 4) Sicurezza e controllo degli accessi insufficienti;

Quali problemi presenta il salvare i dati in un foglio di calcolo?

- 1) Difficile garantire integrità referenziale dei dati;
- 2) Nessun audit trail per registrare le modifiche;
- 3) Scarsa scalabilità in caso di dataset di grandi dimensioni;
- 4) Sicurezza e controllo degli accessi insufficienti;
- 5) Prestazioni limitate in fase di analisi e filtraggio dei dati.

Quali problemi presenta il salvare i dati in un foglio di calcolo?

- 1) Difficile garantire integrità referenziale dei dati;
- 2) Nessun audit trail per registrare le modifiche;
- 3) Scarsa scalabilità in caso di dataset di grandi dimensioni;
- 4) Sicurezza e controllo degli accessi insufficienti;
- 5) Prestazioni limitate in fase di analisi e filtraggio dei dati.

Soluzioni?

Quali problemi presenta il salvare i dati in un foglio di calcolo?

- 1) Difficile garantire integrità referenziale dei dati;
- 2) Nessun audit trail per registrare le modifiche;
- 3) Scarsa scalabilità in caso di dataset di grandi dimensioni;
- 4) Sicurezza e controllo degli accessi insufficienti;
- 5) Prestazioni limitate in fase di analisi e filtraggio dei dati.

Soluzioni?

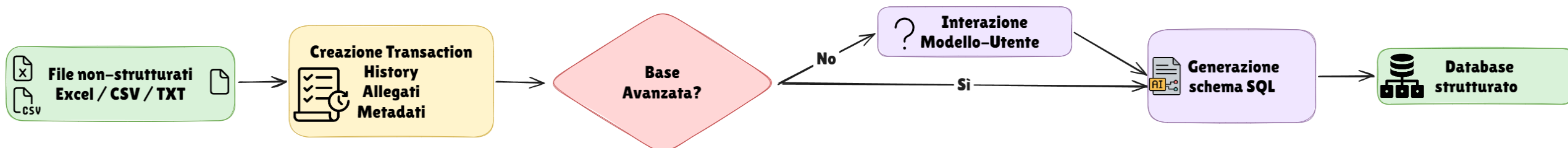
Realizzare un software che permetta di convertire i fogli di calcolo (dati semi-strutturati) in database (dati strutturati)

Quali problemi presenta il salvare i dati in un foglio di calcolo?

- 1) Difficile garantire integrità referenziale dei dati;
- 2) Nessun audit trail per registrare le modifiche;
- 3) Scarsa scalabilità in caso di dataset di grandi dimensioni;
- 4) Sicurezza e controllo degli accessi insufficienti;
- 5) Prestazioni limitate in fase di analisi e filtraggio dei dati.

Soluzioni?

Realizzare un software che permetta di convertire i fogli di calcolo (dati semi-strutturati) in database (dati strutturati)



Design Pattern MVP

Design Pattern MVP

- Separazione Model-View-Presenter;

Design Pattern MVP

- Separazione Model-View-Presenter;
- View passiva e logica nel Presenter;

Design Pattern MVP

- Separazione Model-View-Presenter;
- View passiva e logica nel Presenter;
- Maggiore testabilità e modularità.

Design Pattern MVP

- Separazione Model-View-Presenter;
- View passiva e logica nel Presenter;
- Maggiore testabilità e modularità.



Highlight.js

Highlight.js

- Regex ottimizzate per l'evidenziazione automatica della sintassi.

SheetJS

SheetJS

- Gestione file Excel/CSV;

SheetJS

- Gestione file Excel/CSV;
- Conversione dati in fogli di calcolo.

Architettura Client-Server

Architettura Client-Server

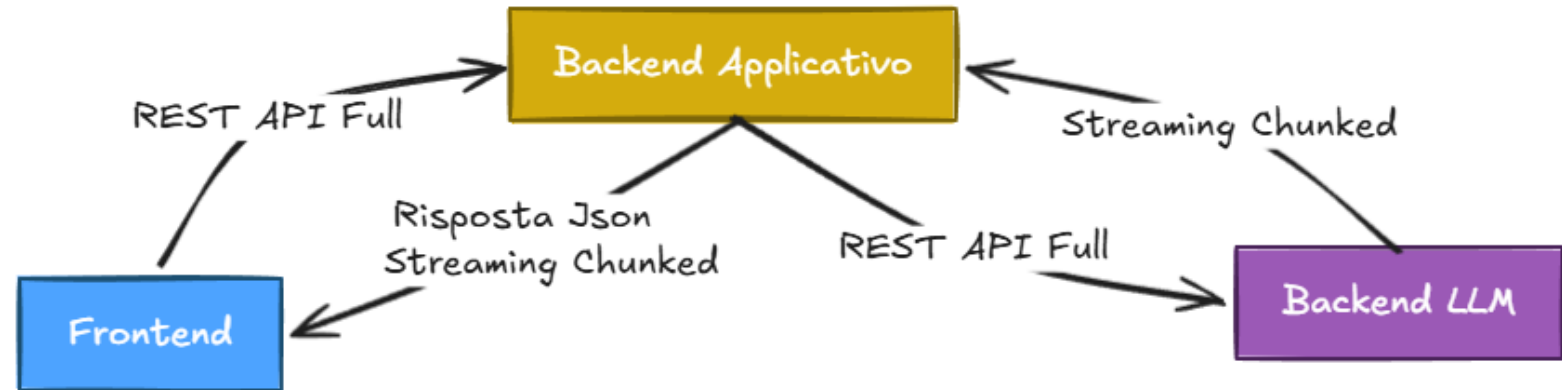
- API restFul;

Architettura Client-Server

- **API restFul;**
- **Separazione delle responsabilità.**

Architettura Client-Server

- API restFul;
- Separazione delle responsabilità.



Chunked Transfer Encoding

Chunked Transfer Encoding

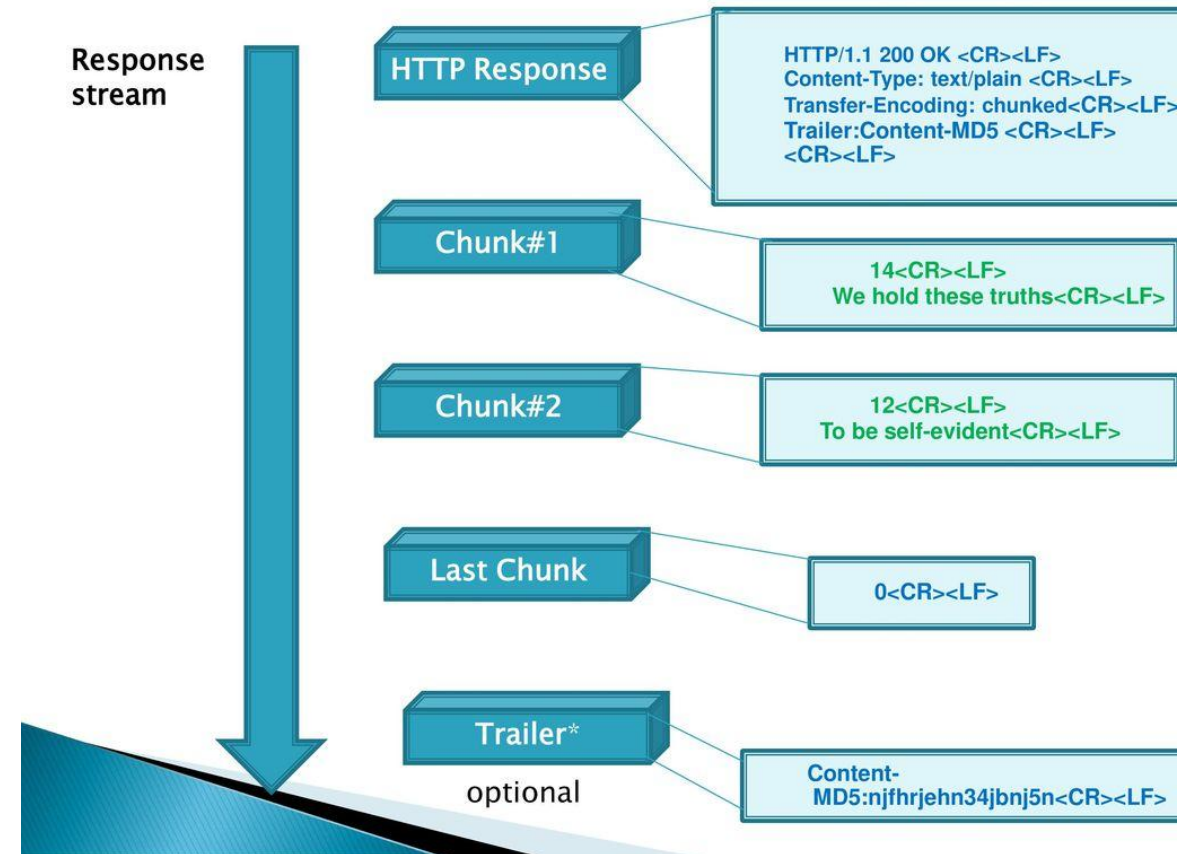
- Streaming dei dati senza Content-Length;

Chunked Transfer Encoding

- Streaming dei dati senza Content-Length;
- Utile per token streaming LLM.

Chunked Transfer Encoding

- Streaming dei dati senza Content-Length;
- Utile per token streaming LLM.



Backend LLM

Backend LLM

- Ollama per inferenza locale;

Backend LLM

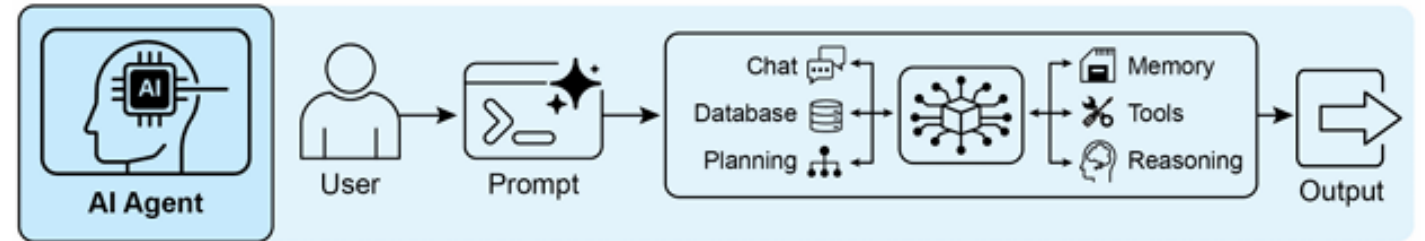
- Ollama per inferenza locale;
- Pattern Agentic per l'orchestrazione;

Backend LLM

- Ollama per inferenza locale;
- Pattern Agentic per l'orchestrazione;
- Modello Qwen2.5-Coder-7B-Instruct.

Backend LLM

- Ollama per inferenza locale;
- Pattern Agentic per l'orchestrazione;
- Modello Qwen2.5-Coder-7B-Instruct.



Quali categorie di strumenti esistono?

Quali categorie di strumenti esistono?

- Convertitori Excel-to-SQL;

Quali categorie di strumenti esistono?

- Convertitori Excel-to-SQL;
- LIMS (Laboratory Information Management System);

Quali categorie di strumenti esistono?

- Convertitori Excel-to-SQL;
- LIMS (Laboratory Information Management System);
- ELN (Electronic Lab Notebook);

Quali categorie di strumenti esistono?

- Convertitori Excel-to-SQL;
- LIMS (Laboratory Information Management System);
- ELN (Electronic Lab Notebook);
- GMOD.

Convertitori Excel-to-SQL

Convertitori Excel-to-SQL

- Conversione sintattica fogli di calcolo -> SQL;

Convertitori Excel-to-SQL

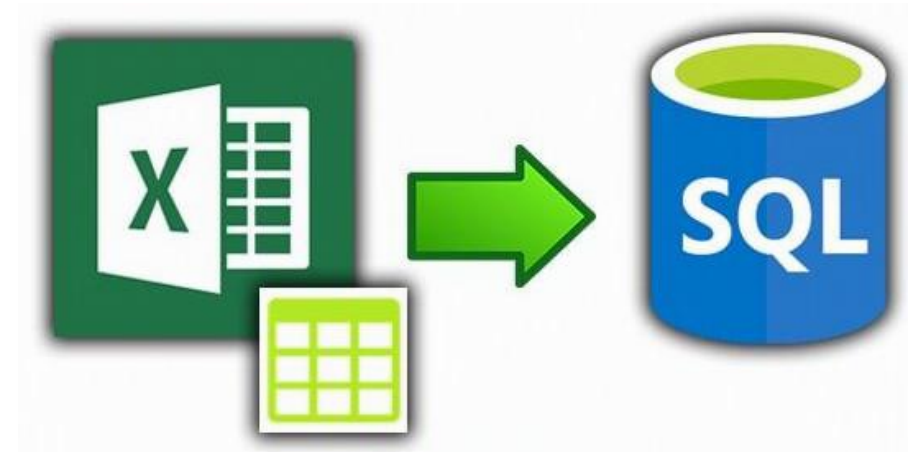
- Conversione sintattica fogli di calcolo -> SQL;
- **Nessuna comprensione semantica;**

Convertitori Excel-to-SQL

- Conversione sintattica fogli di calcolo -> SQL;
- **Nessuna comprensione semantica;**
- **Nessuna normalizzazione.**

Convertitori Excel-to-SQL

- Conversione sintattica fogli di calcolo -> SQL;
- **Nessuna comprensione semantica;**
- **Nessuna normalizzazione.**



LIMS (Senaite)

LIMS (Senaite)

- Gestione completa del ciclo del campione;

LIMS (Senaite)

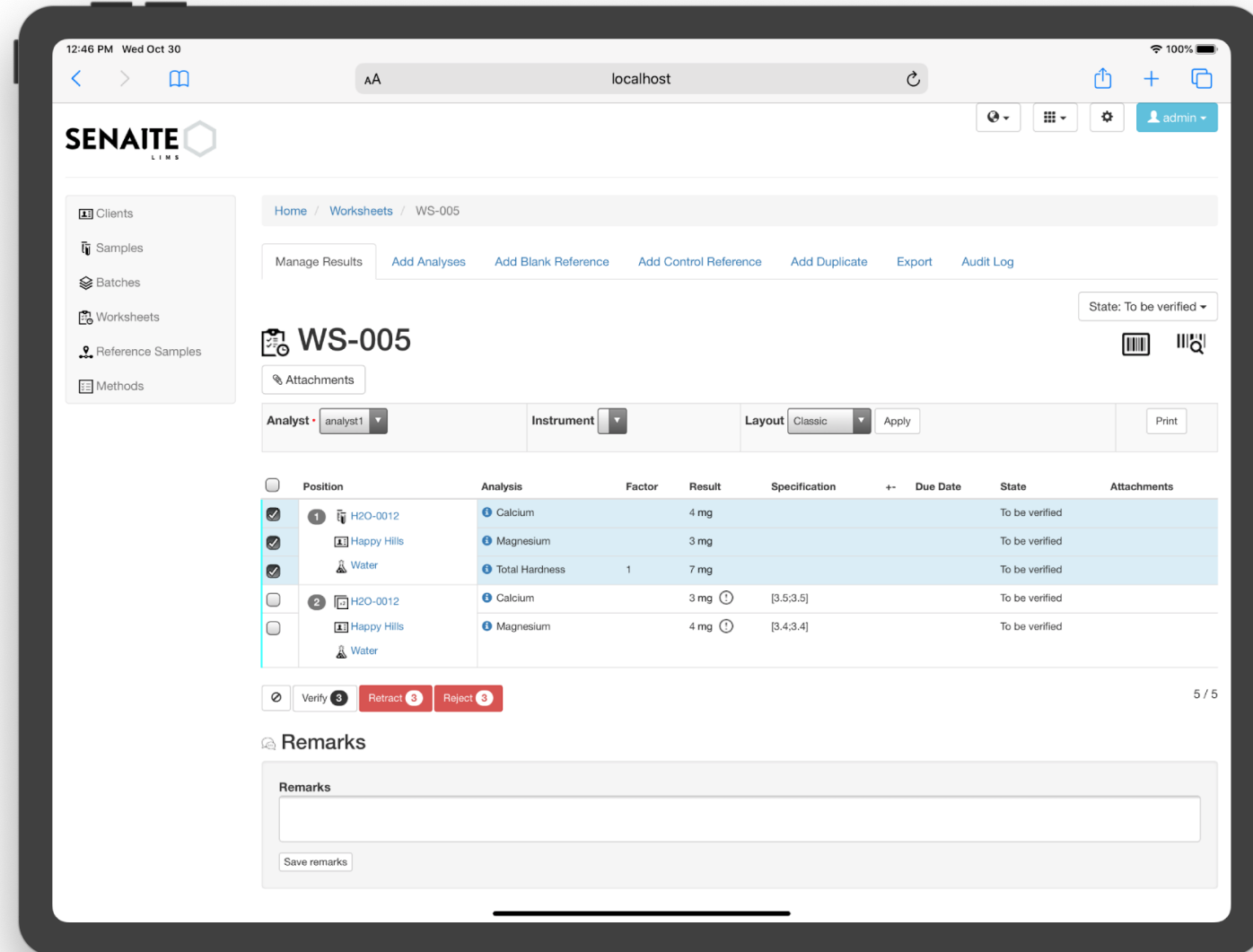
- **Gestione completa del ciclo del campione;**
- **Audit trail;**

LIMS (Senaite)

- **Gestione completa del ciclo del campione;**
- **Audit trail;**
- **Funzionalità di gestione avanzate.**

LIMS (Senaite)

- Gestione completa del ciclo del campione;
- Audit trail;
- Funzionalità di gestione avanzate.



ELN (SciNote)

ELN (SciNote)

- Documentazione esperimenti;

ELN (SciNote)

- Documentazione esperimenti;
- File Manager;

ELN (SciNote)

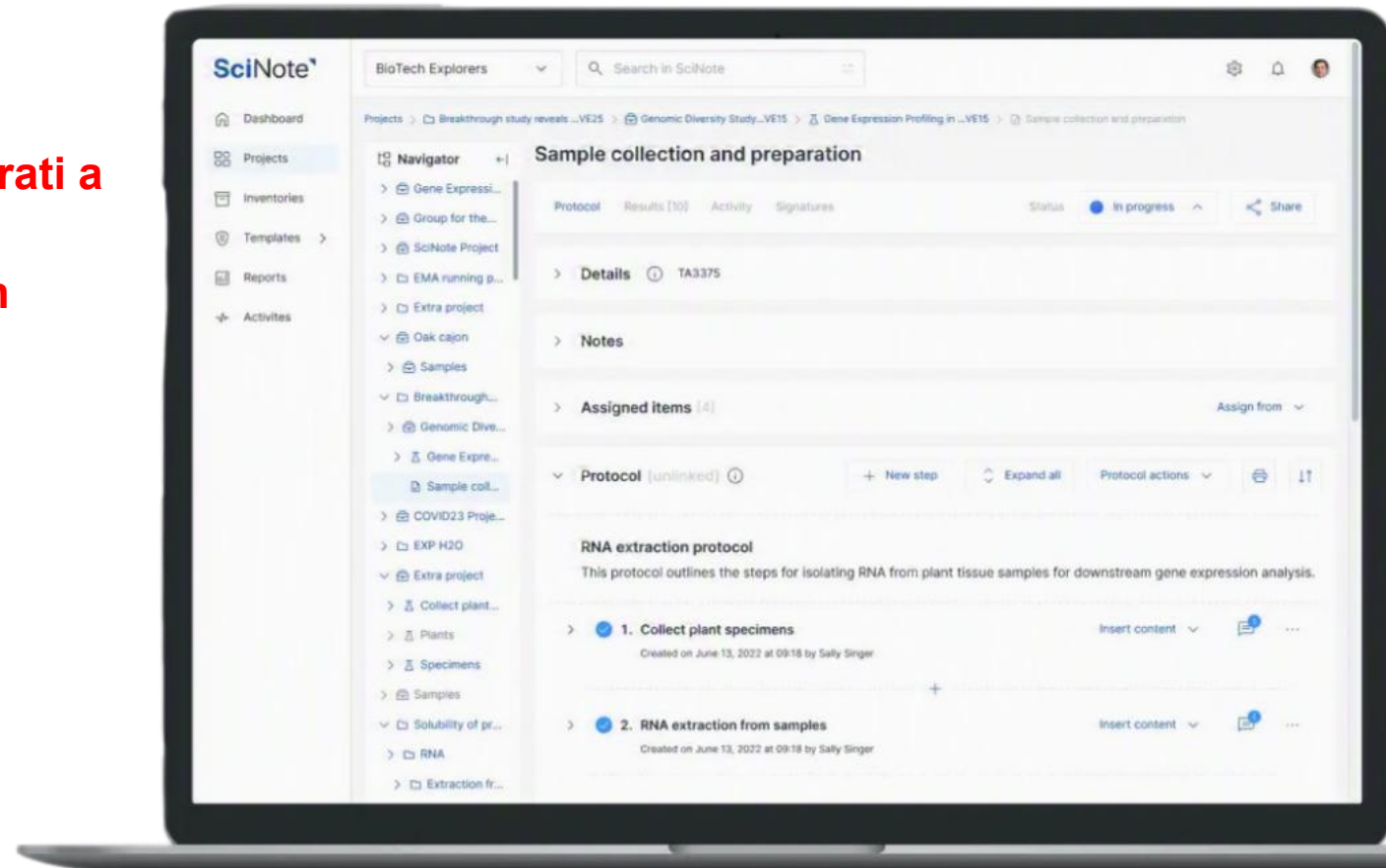
- Documentazione esperimenti;
- File Manager;
- **Complessa migrazione da dati non/semi-strutturati a dati strutturati;**

ELN (SciNote)

- Documentazione esperimenti;
- File Manager;
- Complessa migrazione da dati non/semi-strutturati a dati strutturati;
- Complesso adattamento processi di utilizzo con fogli di calcolo.

ELN (SciNote)

- Documentazione esperimenti;
- File Manager;
- Complessa migrazione da dati non/semi-strutturati a dati strutturati;
- Complesso adattamento processi di utilizzo con fogli di calcolo.



GMOD (Chado, Tripal, Jbrowse, Apollo)

GMOD (Chado, Tripal, Jbrowse, Apollo)

- Ecosistema di software per gestione di dati biologici complessi;

GMOD (Chado, Tripal, Jbrowse, Apollo)

- Ecosistema di software per gestione di dati biologici complessi;
- Richiede dati già strutturati;

GMOD (Chado, Tripal, Jbrowse, Apollo)

- Ecosistema di software per gestione di dati biologici complessi;
- Richiede dati già strutturati;
- Impossibilità di importare dati utilizzando un LLM;

GMOD (Chado, Tripal, Jbrowse, Apollo)

- Ecosistema di software per gestione di dati biologici complessi;
- Richiede dati già strutturati;
- Impossibilità di importare dati utilizzando un LLM;
- Richieste competenze avanzate per la gestione.

GMOD (Chado, Tripal, Jbrowse, Apollo)

- Ecosistema di software per gestione di dati biologici complessi;
- Richiede dati già strutturati;
- Impossibilità di importare dati utilizzando un LLM;
- Richieste competenze avanzate per la gestione.



bioCMS vs Excel-to-SQL

bioCMS vs Excel-to-SQL

- Comprensione semantica mediante LLM;

bioCMS vs Excel-to-SQL

- **Comprensione semantica mediante LLM;**
- **Generazione automatica di schemi SQL in 3FN;**

bioCMS vs Excel-to-SQL

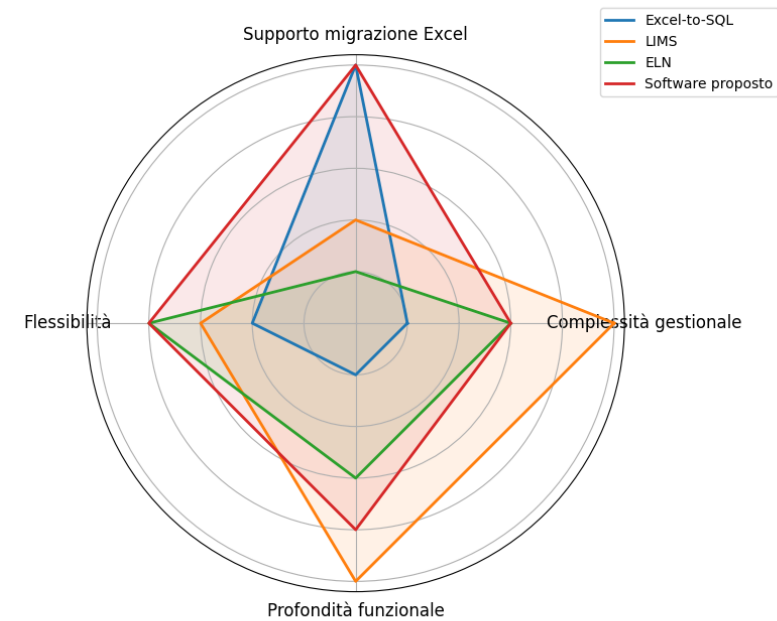
- **Comprensione semantica mediante LLM;**
- **Generazione automatica di schemi SQL in 3FN;**
- **Gestione Utenti;**

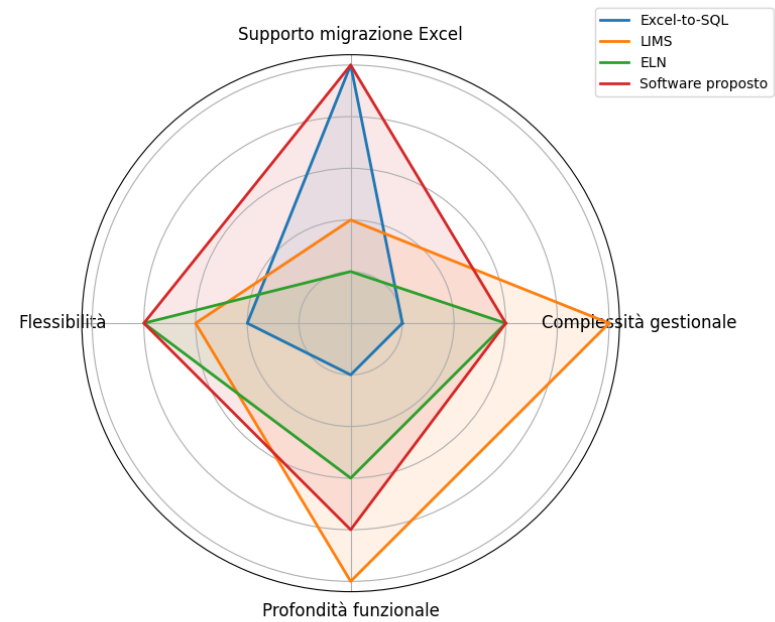
bioCMS vs Excel-to-SQL

- **Comprensione semantica mediante LLM;**
- **Generazione automatica di schemi SQL in 3FN;**
- **Gestione Utenti;**
- **Audit trail.**

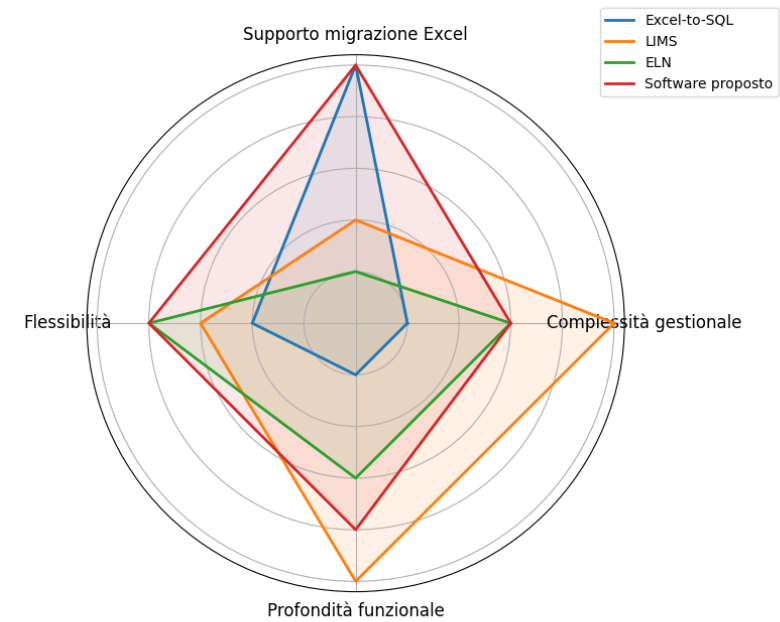
bioCMS vs Excel-to-SQL

- **Comprensione semantica mediante LLM;**
- **Generazione automatica di schemi SQL in 3FN;**
- **Gestione Utenti;**
- **Audit trail.**



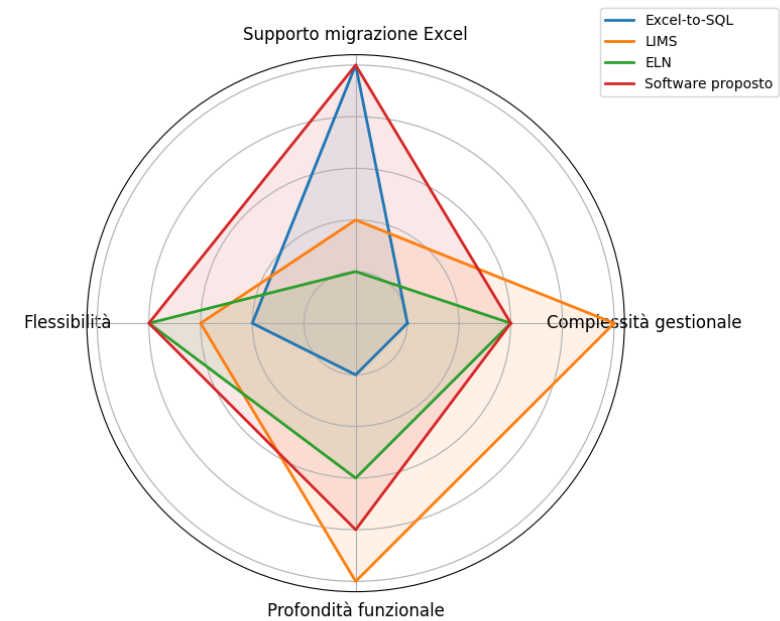


bioCMS vs LIMS



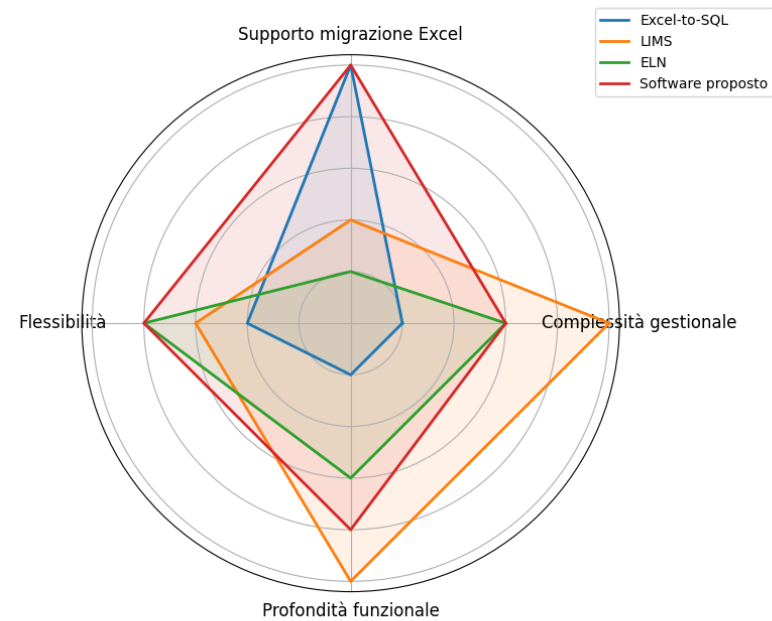
bioCMS vs LIMS

- Soluzione più semplice con meno funzionalità avanzate inutilizzate;



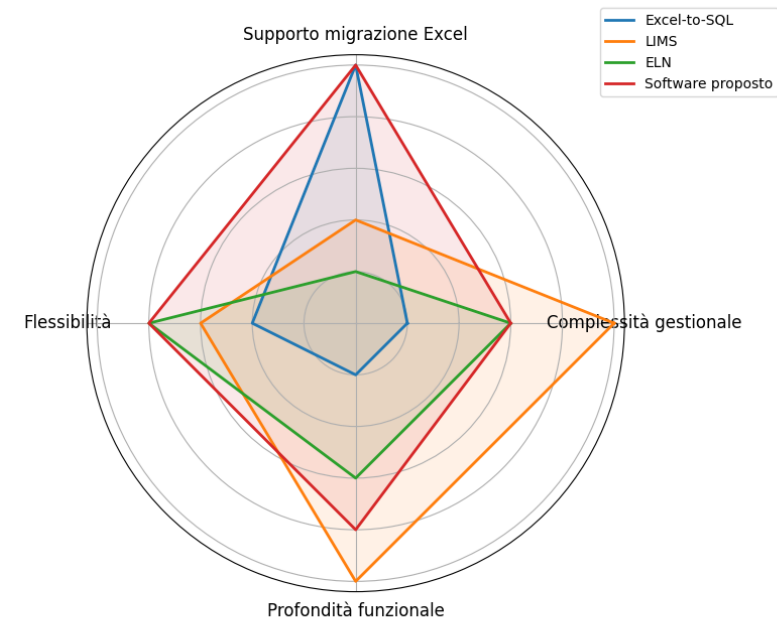
bioCMS vs LIMS

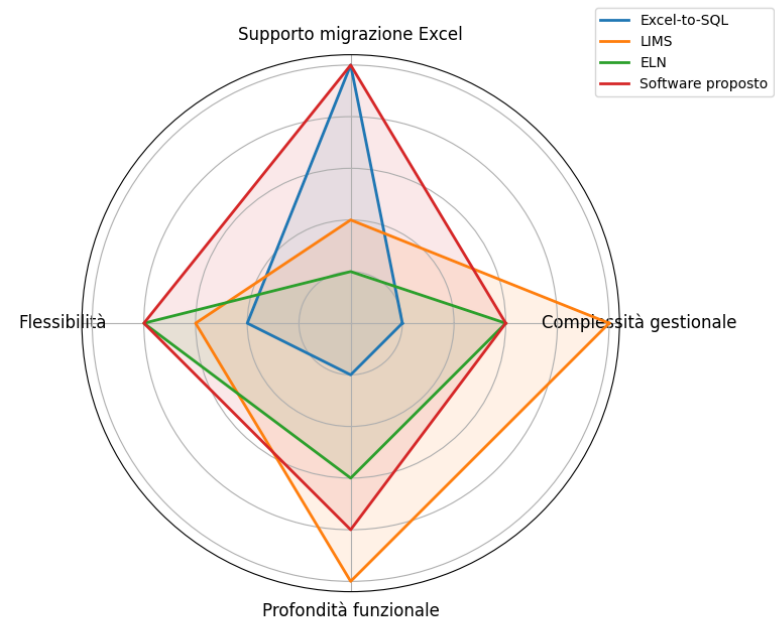
- Soluzione più semplice con meno funzionalità avanzate inutilizzate;
- Focalizzazione sulla conversione da dati non/semi-strutturati in dati strutturati;



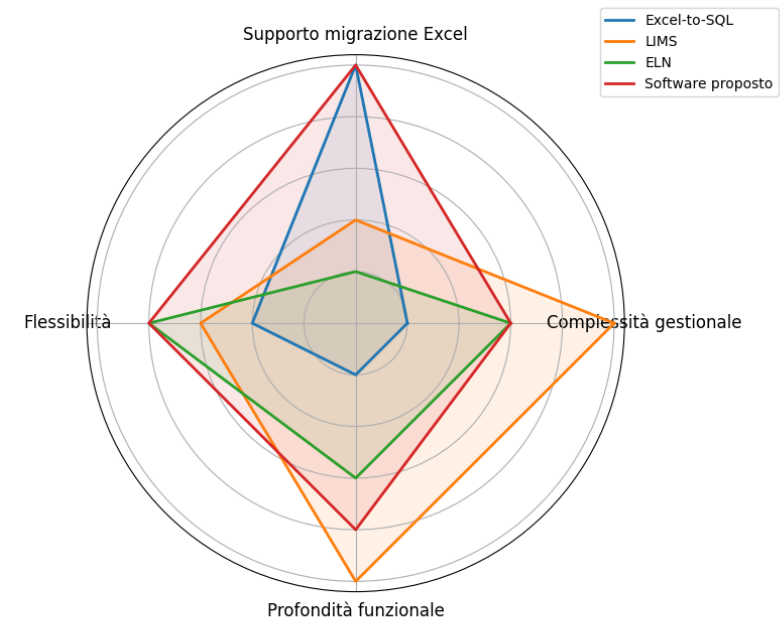
bioCMS vs LIMS

- Soluzione più semplice con meno funzionalità avanzate inutilizzate;
- Focalizzazione sulla conversione da dati non/semi-strutturati in dati strutturati;
- **Mancanza di workflow automatici sui dati importati.**



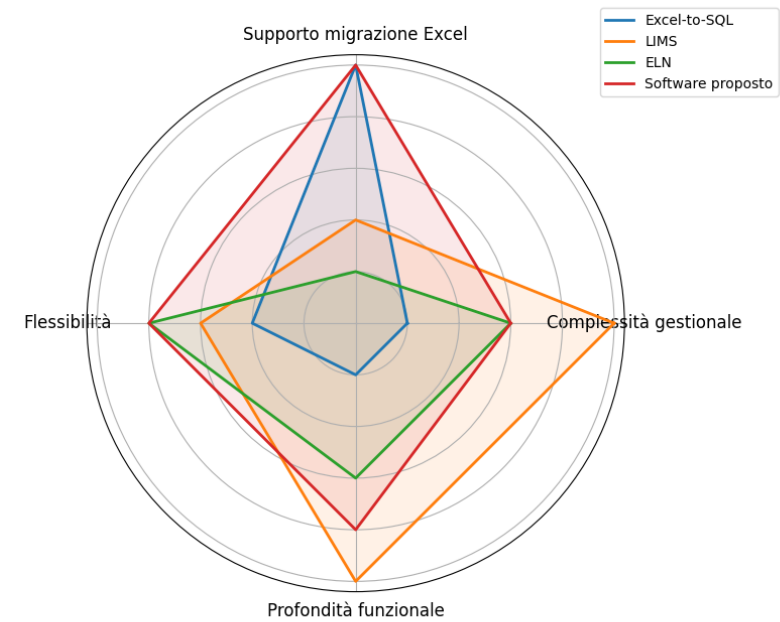


bioCMS vs ELN



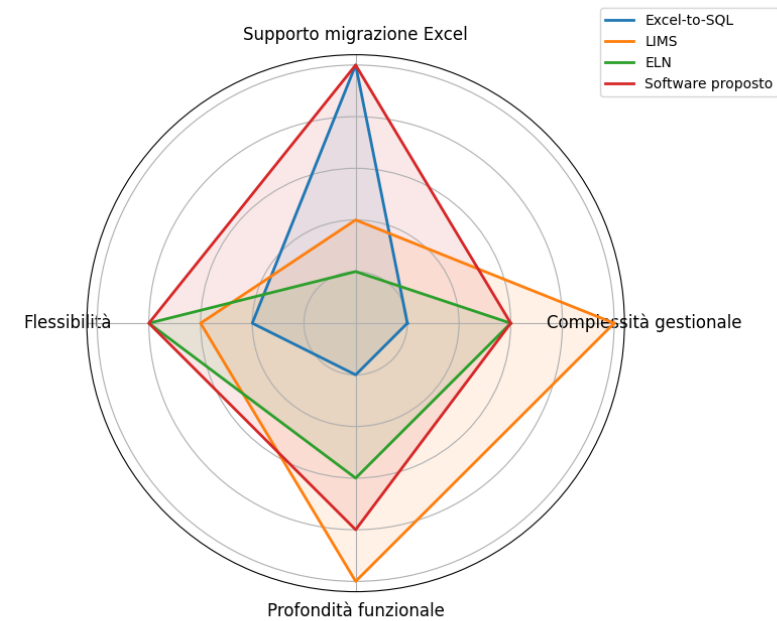
bioCMS vs ELN

- Conversione completa dei dati non/semi-strutturati in dati strutturati;



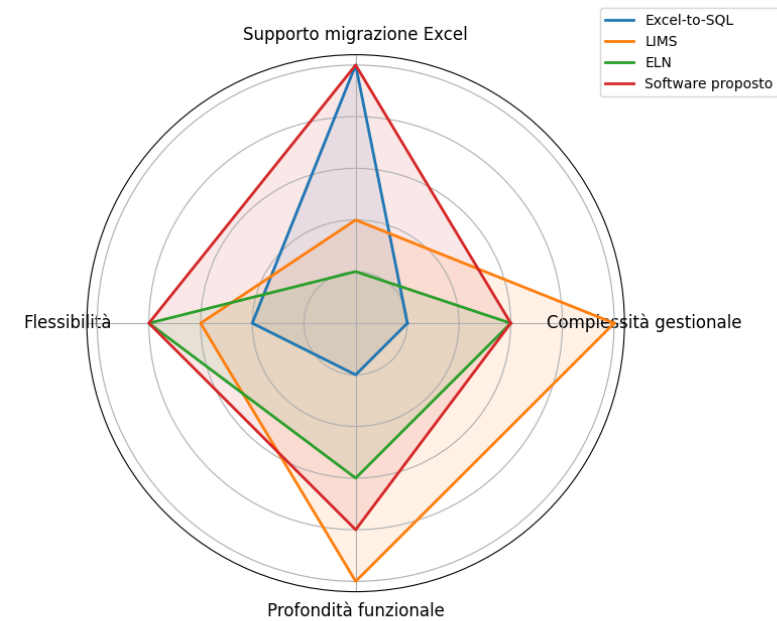
bioCMS vs ELN

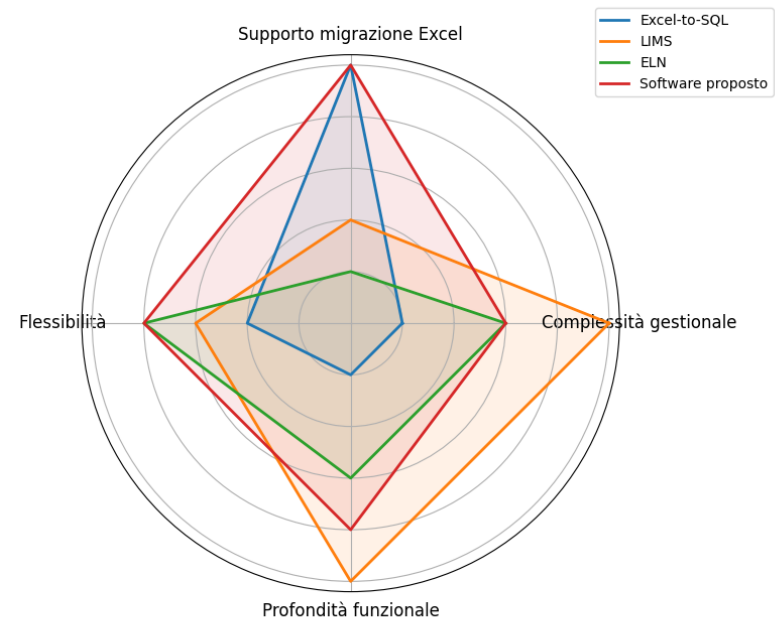
- Conversione completa dei dati non/semi-strutturati in dati strutturati;
- Mancanza di funzionalità avanzata di gestione file;



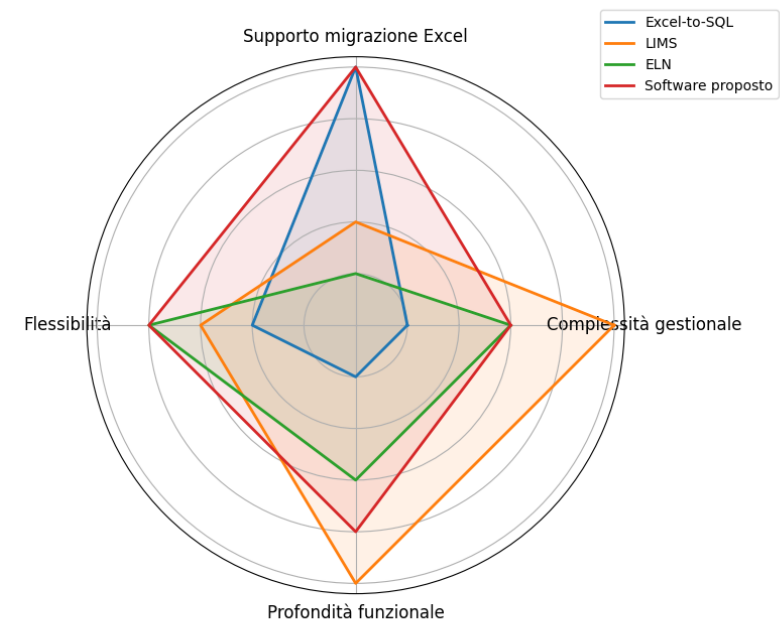
bioCMS vs ELN

- Conversione completa dei dati non/semi-strutturati in dati strutturati;
- Mancanza di funzionalità avanzata di gestione file;
- Mancanza della documentazione degli esperimenti.



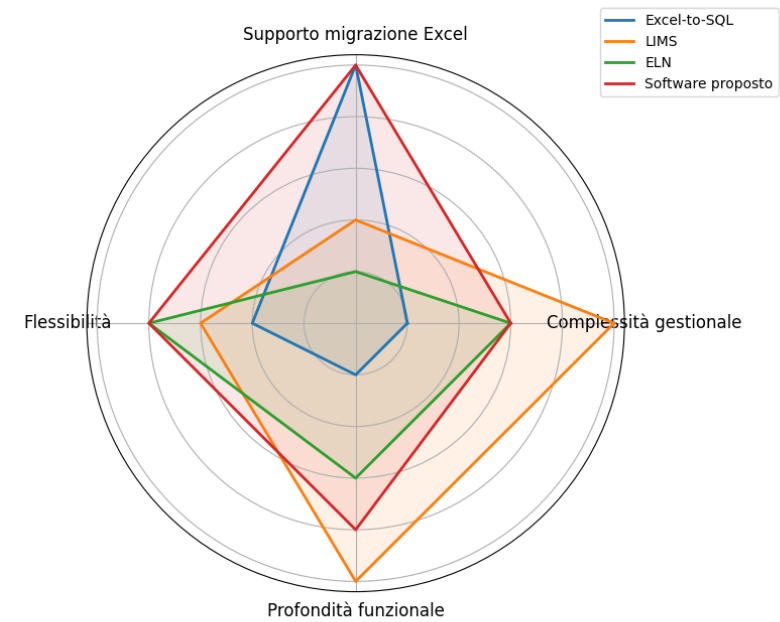


bioCMS vs GMOD



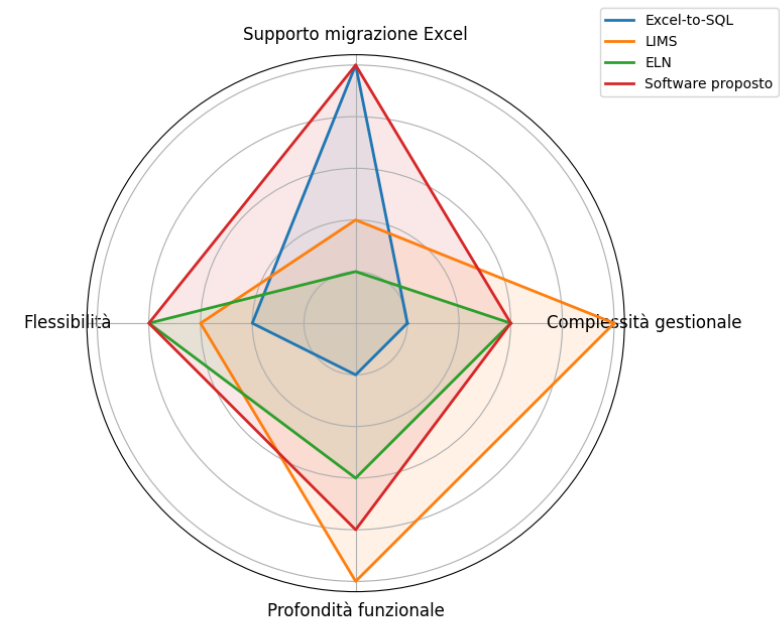
bioCMS vs GMOD

- Realizzato per dati microbiologici tabellari;



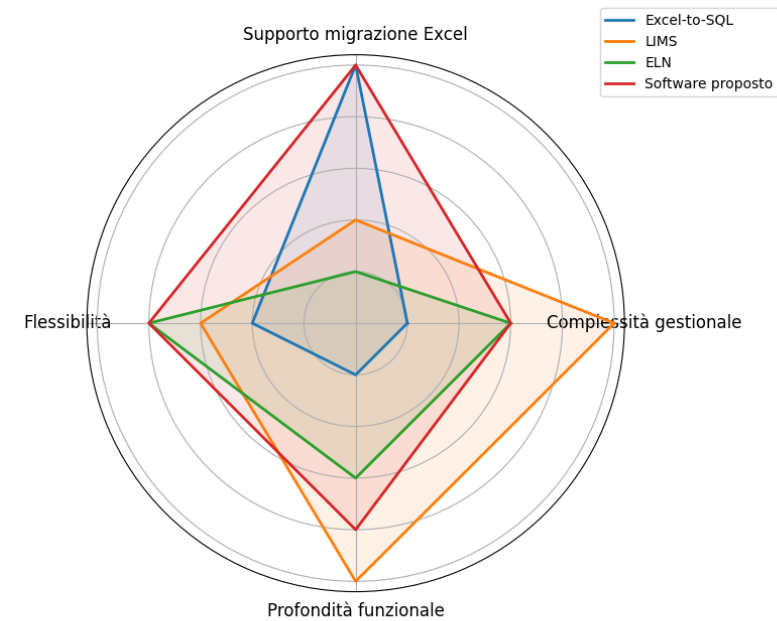
bioCMS vs GMOD

- Realizzato per dati microbiologici tabellari;
- Non richiede modelli genomici;



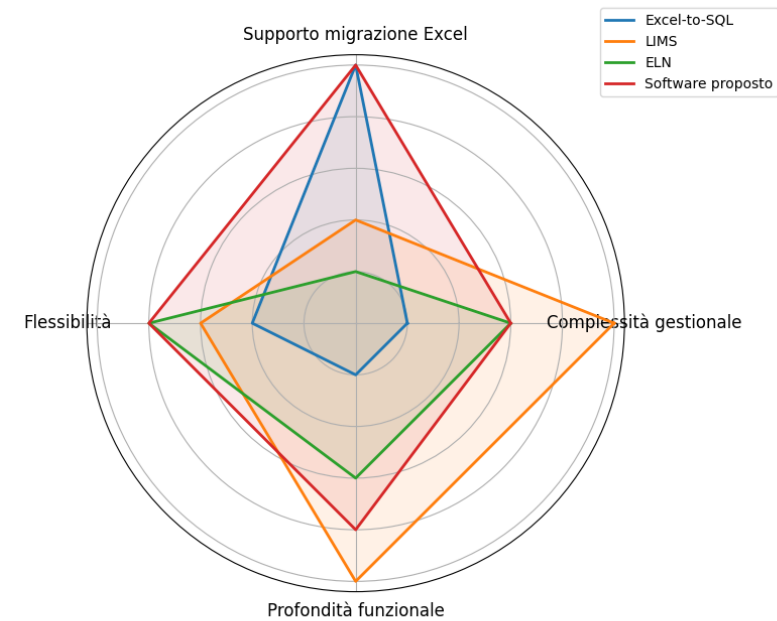
bioCMS vs GMOD

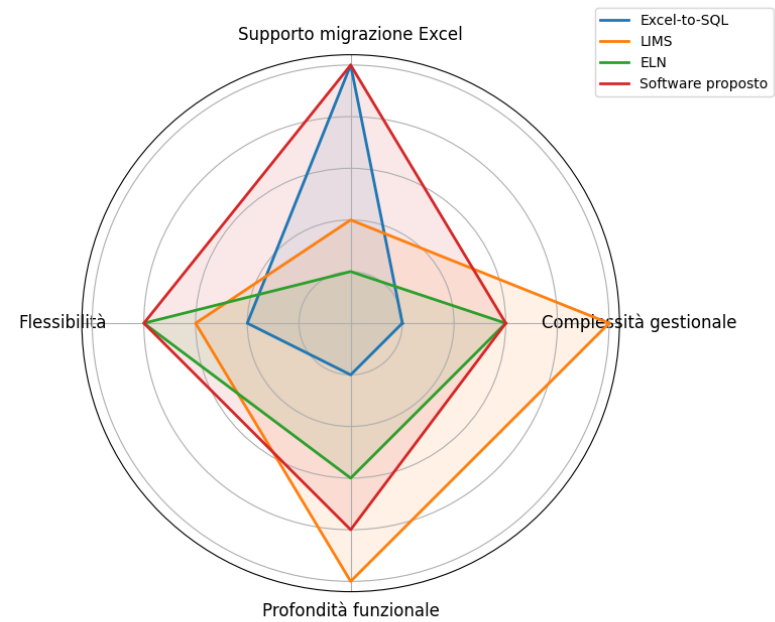
- Realizzato per dati microbiologici tabellari;
- Non richiede modelli genomici;
- Integrazione nativa agli LLM;



bioCMS vs GMOD

- Realizzato per dati microbiologici tabellari;
- Non richiede modelli genomici;
- Integrazione nativa agli LLM;
- Realizzazione di vetrine.





Containerizzazione ed ambiente di sviluppo

Containerizzazione ed ambiente di sviluppo

- Docker per portabilità, isolamento e replicabilità dell'ambiente;

Containerizzazione ed ambiente di sviluppo

- Docker per portabilità, isolamento e replicabilità dell'ambiente;
- Dockerfile basato su Ubuntu 26.04 con Apache+PHP, MariaDB, MongoDB, Python e Ollama;

Containerizzazione ed ambiente di sviluppo

- Docker per portabilità, isolamento e replicabilità dell'ambiente;
- Dockerfile basato su Ubuntu 26.04 con Apache+PHP, MariaDB, MongoDB, Python e Ollama;
- Docker-Compose per orchestrazione dei servizi e gestione delle variabili d'ambiente.

Containerizzazione ed ambiente di sviluppo

- Docker per portabilità, isolamento e replicabilità dell'ambiente;
- Dockerfile basato su Ubuntu 26.04 con Apache+PHP, MariaDB, MongoDB, Python e Ollama;
- Docker-Compose per orchestrazione dei servizi e gestione delle variabili d'ambiente.

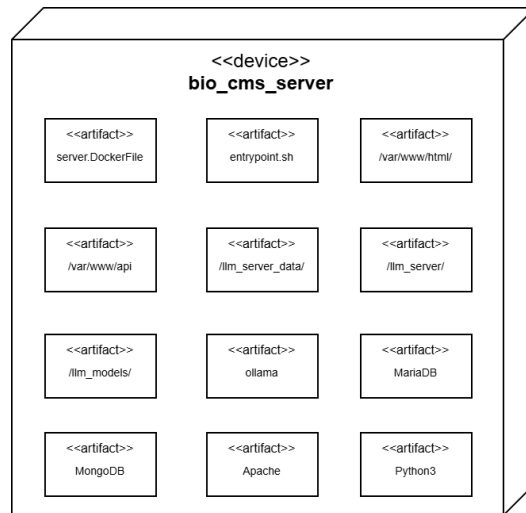


Diagramma di distribuzione con ollama installato sul container.

Containerizzazione ed ambiente di sviluppo

- Docker per portabilità, isolamento e replicabilità dell'ambiente;
- Dockerfile basato su Ubuntu 26.04 con Apache+PHP, MariaDB, MongoDB, Python e Ollama;
- Docker-Compose per orchestrazione dei servizi e gestione delle variabili d'ambiente.

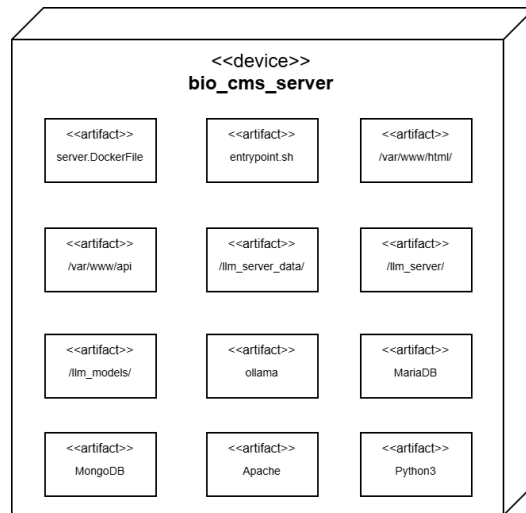


Diagramma di distribuzione con ollama installato sul container.

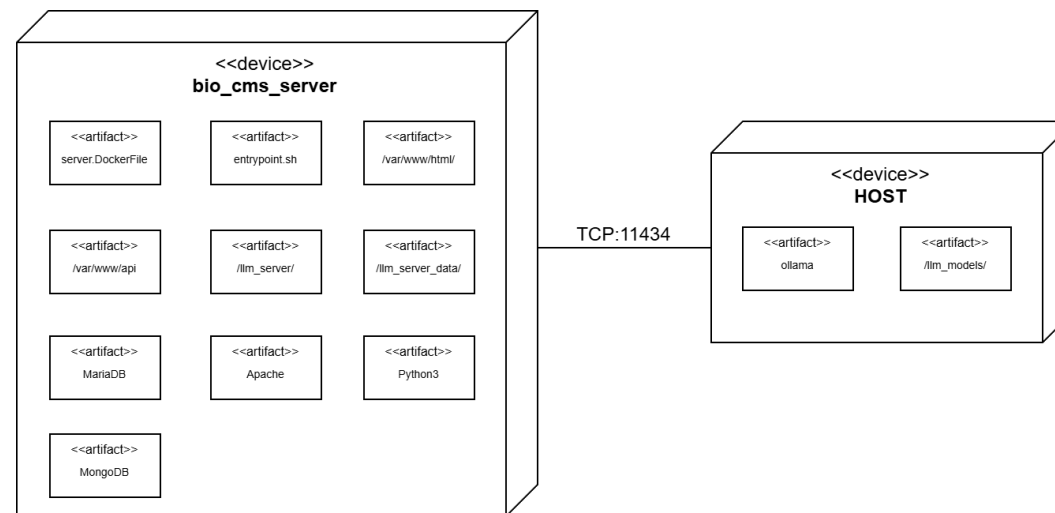


Diagramma di distribuzione con ollama installato sull'host.

Configurazione di avvio

Configurazione di avvio

- Apache con virtualHost;

Configurazione di avvio

- **Apache con virtualHost;**
- **Script di setup automatico per MariaDB e MongoDB;**

Configurazione di avvio

- **Apache con virtualHost;**
- **Script di setup automatico per MariaDB e MongoDB;**
- **Ambiente python isolato configurato per l'utilizzo di Flask, Pydantic, Pandas e Ollama.**

Backend LLM

Backend LLM

- Server Flask con blueprint modulari;

Backend LLM

- Server Flask con blueprint modulari;
- ChatEngine: orchestrazione completa del ciclo LLM (streaming, tool calling, validazione JSON);

Backend LLM

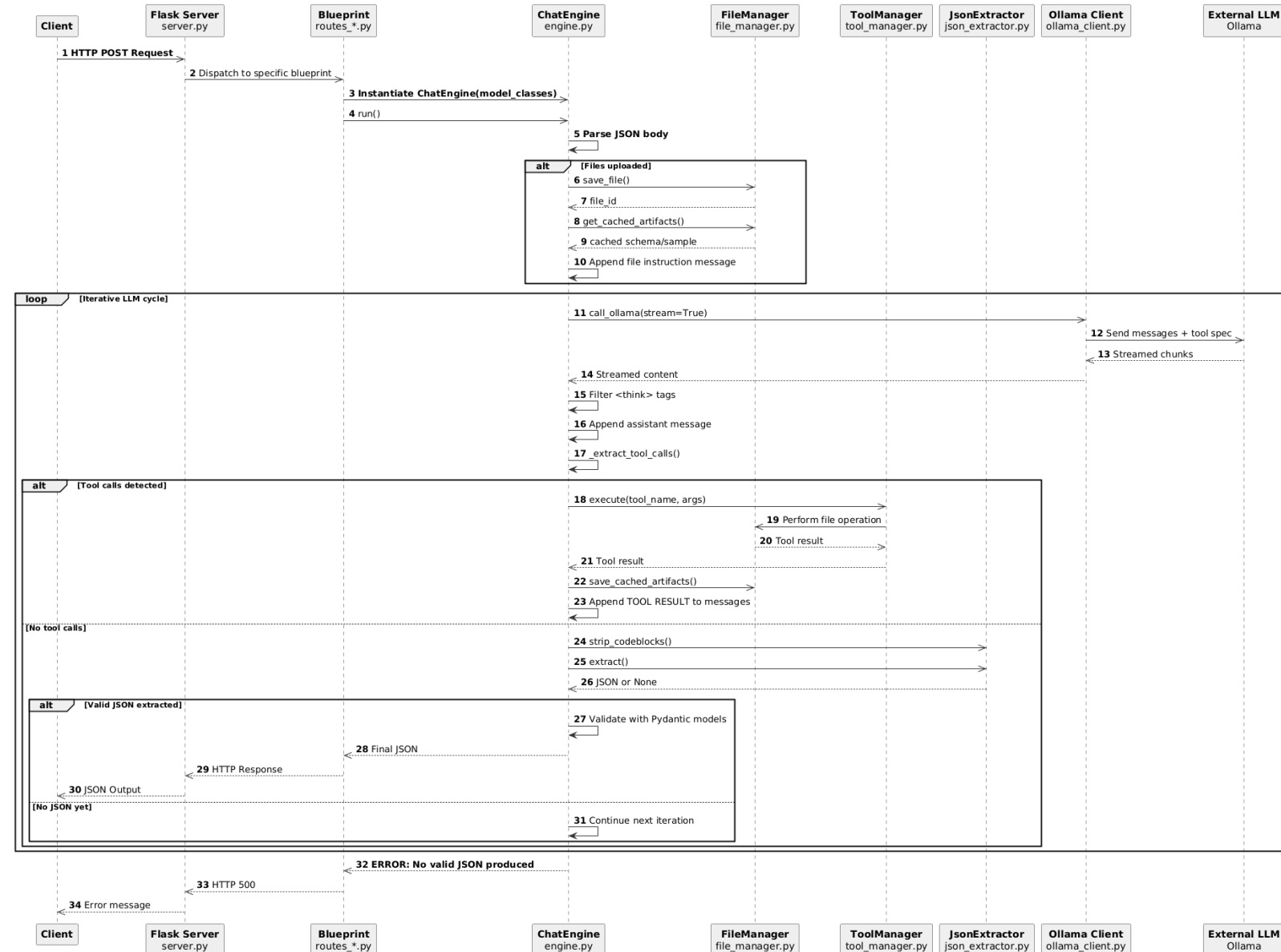
- **Server Flask con blueprint modulari;**
- **ChatEngine: orchestrazione completa del ciclo LLM (streaming, tool calling, validazione JSON);**
- **FileManager + ToolManager: gestione file, caching, inferenza schema e sample CSV;**

Backend LLM

- **Server Flask con blueprint modulari;**
- **ChatEngine: orchestrazione completa del ciclo LLM (streaming, tool calling, validazione JSON);**
- **FileManager + ToolManager: gestione file, caching, inferenza schema e sample CSV;**
- **JsonExtractor: estrazione di JSON validi dal reasoning del modello.**

Backend LLM

- Server Flask con blueprint modulari;
- ChatEngine: orchestrazione completa del ciclo LLM (streaming, tool calling, validazione JSON);
- FileManager + ToolManager: gestione file, caching, inferenza schema e sample CSV;
- JsonExtractor: estrazione di JSON validi dal reasoning del modello.



Sistema di logging

Sistema di logging

- Architettura modulare basata su BaseLogger e logger specializzati;

Sistema di logging

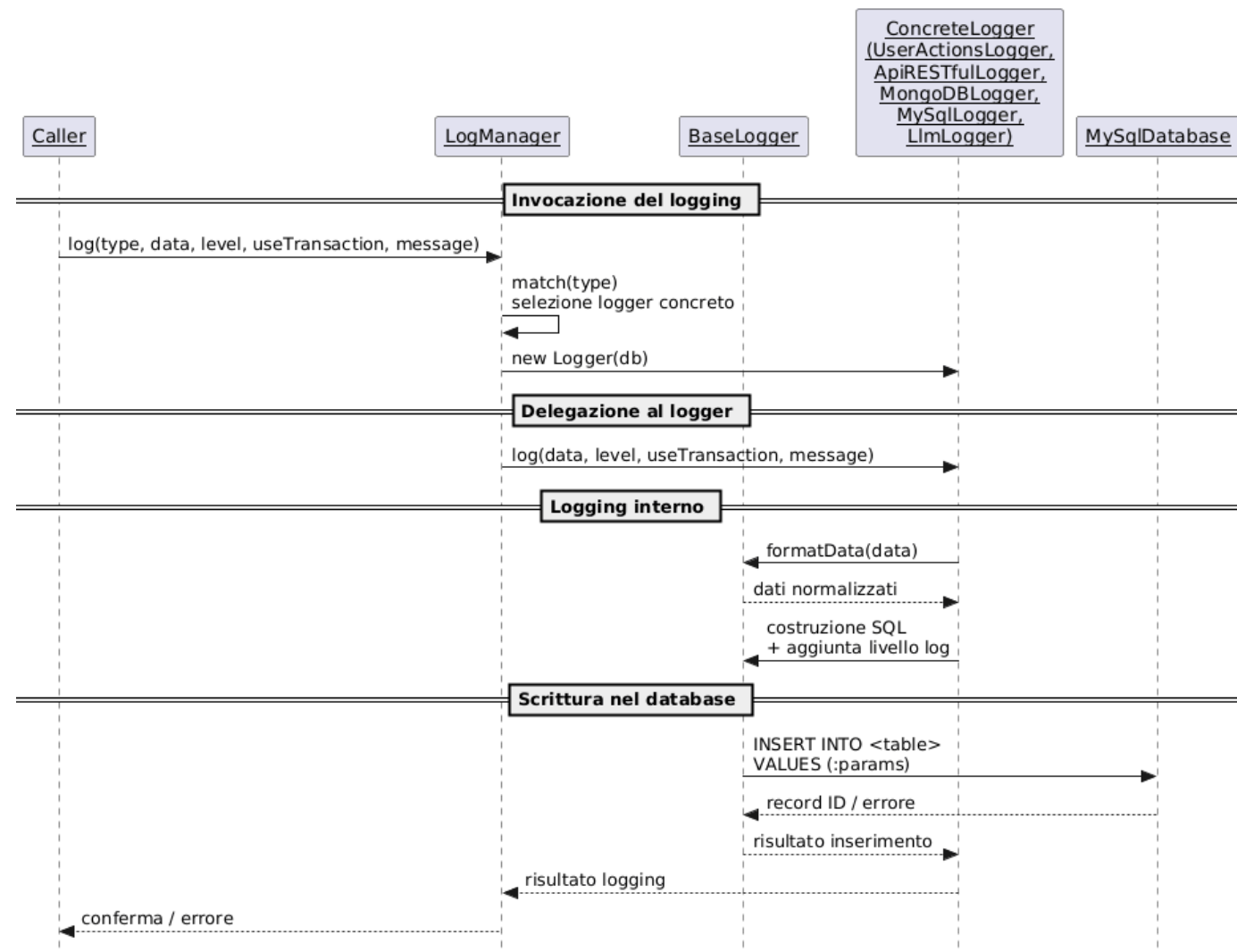
- Architettura modulare basata su BaseLogger e logger specializzati;
- LogManager: selezione dinamica del logger da invocare;

Sistema di logging

- Architettura modulare basata su BaseLogger e logger specializzati;
- LogManager: selezione dinamica del logger da invocare;
- Doppio tracciamento degli errori: database + log nativo PHP.

Sistema di logging

- Architettura modulare basata su BaseLogger e logger specializzati;
- LogManager: selezione dinamica del logger da invocare;
- Doppio tracciamento degli errori: database + log nativo PHP.



Autenticazione e gestione utenti

Autenticazione e gestione utenti

- Configurazione iniziale utente admin;

Autenticazione e gestione utenti

- Configurazione iniziale utente admin;
- Login a due fasi:

Autenticazione e gestione utenti

- Configurazione iniziale utente admin;
- Login a due fasi:
 - 1) Verifica utente, stato password e generazione token JWT intermedio;

Autenticazione e gestione utenti

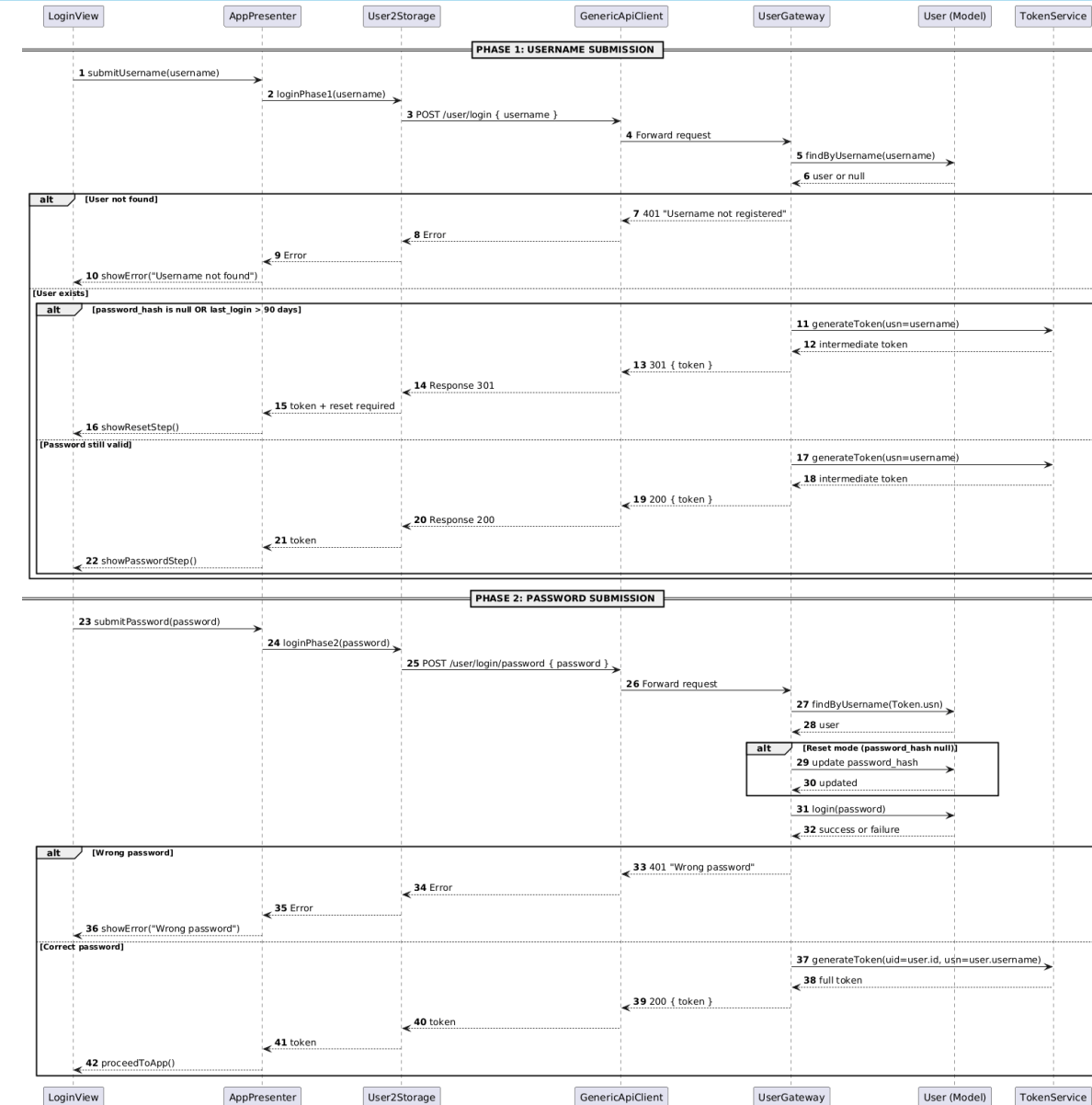
- Configurazione iniziale utente admin;
- Login a due fasi:
 - 1) Verifica utente, stato password e generazione token JWT intermedio;
 - 2) Verifica password, generazione token JWT;

Autenticazione e gestione utenti

- Configurazione iniziale utente admin;
- Login a due fasi:
 - 1) Verifica utente, stato password e generazione token JWT intermedio;
 - 2) Verifica password, generazione token JWT;
- Creazione utente con controlli sui permessi, sicurezza password ed unicità delle username.

Autenticazione e gestione utenti

- Configurazione iniziale utente admin;
- Login a due fasi:
 - 1) Verifica utente, stato password e generazione token JWT intermedio;
 - 2) Verifica password, generazione token JWT;
- Creazione utente con controlli sui permessi, sicurezza password ed unicità delle username.



Gestione permessi

Gestione permessi

- Controllo centralizzato dei permessi mediante il PermissionManager;

Gestione permessi

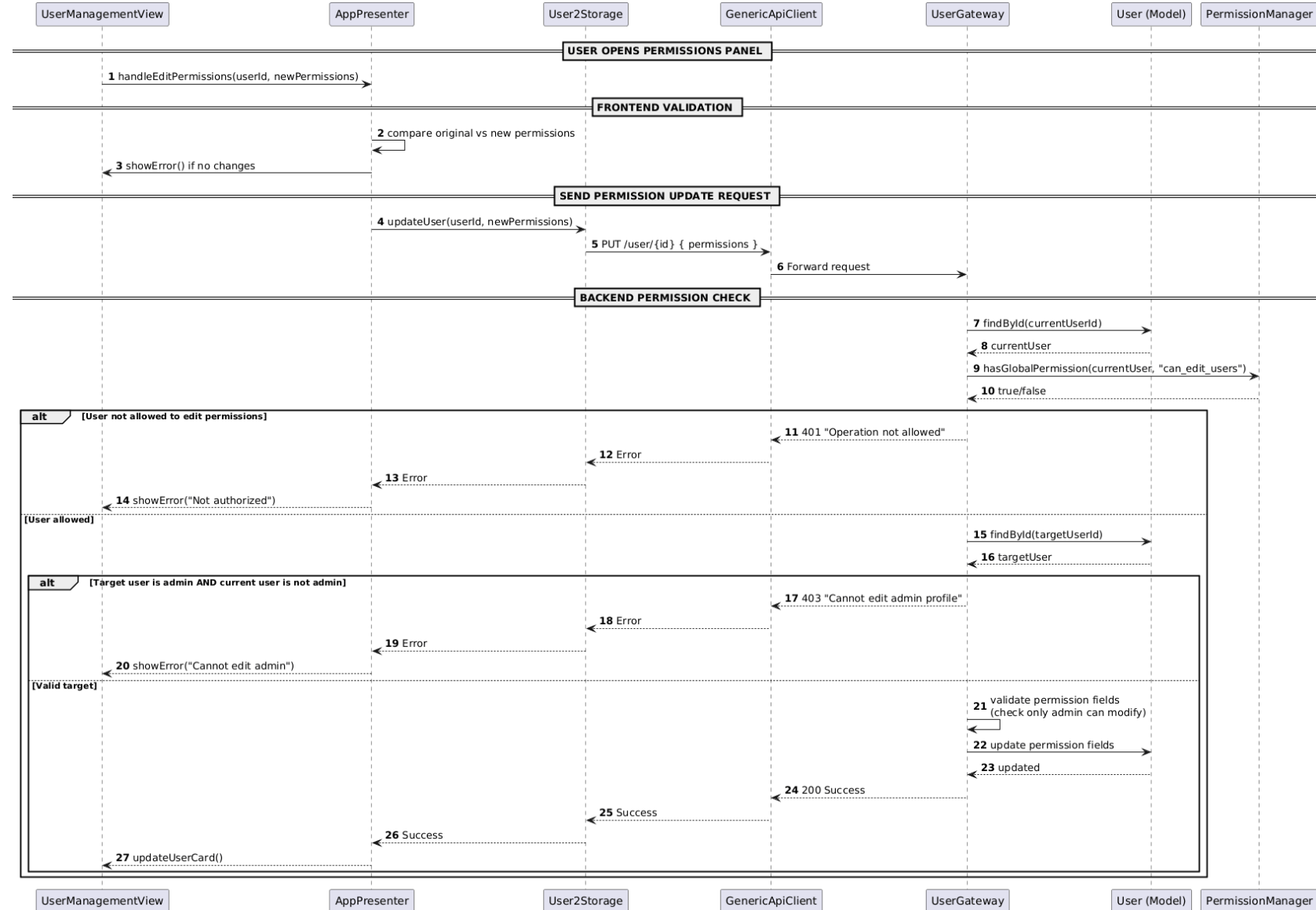
- Controllo centralizzato dei permessi mediante il **PermissionManager**;
- Protezione control escalation dei permessi;

Gestione permessi

- Controllo centralizzato dei permessi mediante il PermissionManager;
- Protezione control escalation dei permessi;
- Frontend adattivo: permette di minimizzare le richieste non consentite all'utente.

Gestione permessi

- Controllo centralizzato dei permessi mediante il **PermissionManager**;
- Protezione control escalation dei permessi;
- Frontend adattivo: permette di minimizzare le richieste non consentite all'utente.



Transazioni

Transazioni

- Creazione transazione con metadati, allegati e stato applicativo;

Transazioni

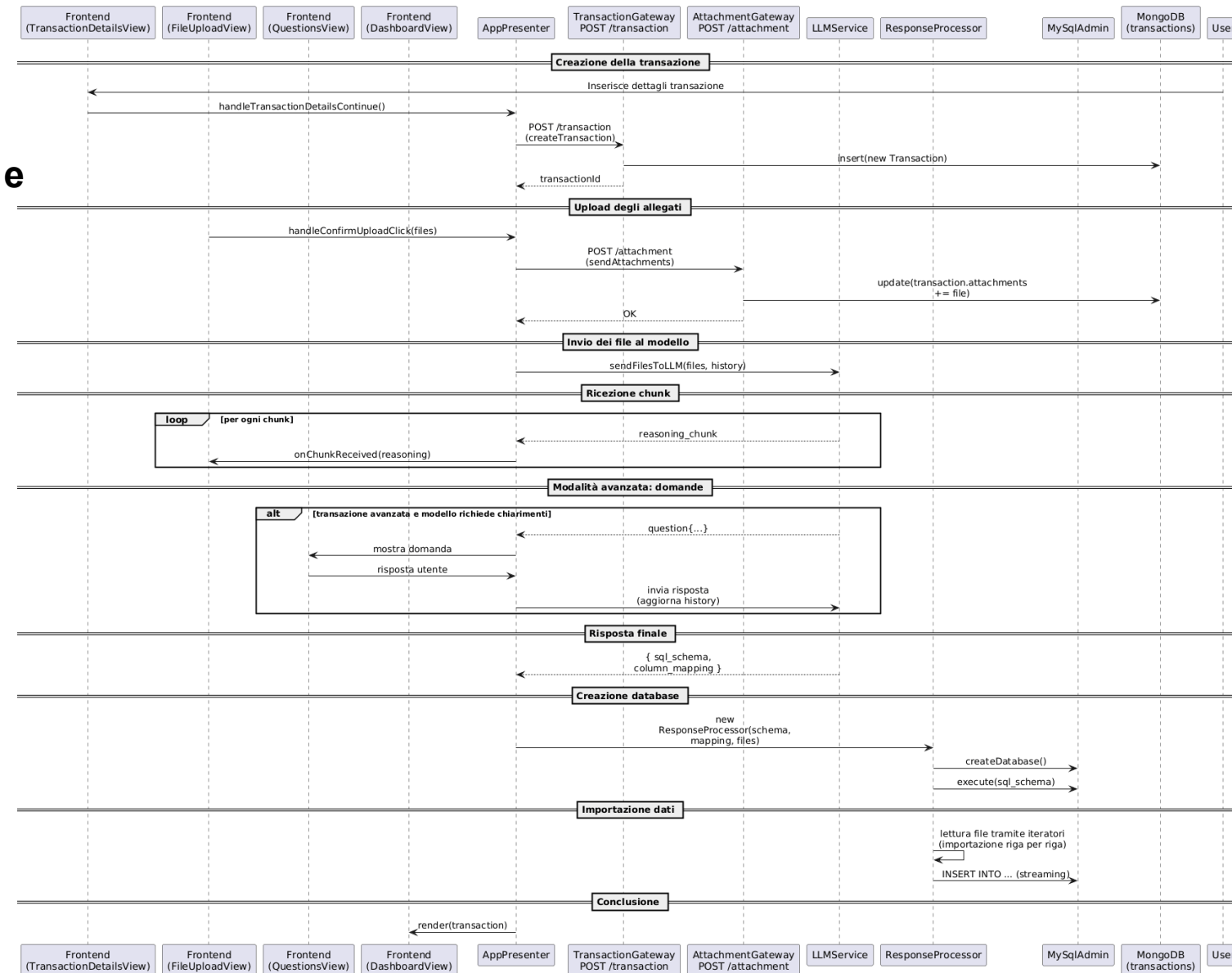
- Creazione transazione con metadati, allegati e stato applicativo;
- Caricamento file, normalizzazione file ed inserimento nella transazione;

Transazioni

- Creazione transazione con metadati, allegati e stato applicativo;
- Caricamento file, normalizzazione file ed inserimento nella transazione;
- Modalità avanzata: dialogo iterativo modello-utente con domande chiarificatrici.

Transazioni

- Creazione transazione con metadati, allegati e stato applicativo;
- Caricamento file, normalizzazione file ed inserimento nella transazione;
- Modalità avanzata: dialogo iterativo modello-utente con domande chiarificatrici.



Elaborazione LLM e generazione database

Elaborazione LLM e generazione database

- Streaming reasoning in tempo reale verso il frontend;

Elaborazione LLM e generazione database

- Streaming reasoning in tempo reale verso il frontend;
- Risposta finale strutturata con `sql_schema` e `column_mapping`;

Elaborazione LLM e generazione database

- Streaming reasoning in tempo reale verso il frontend;
- Risposta finale strutturata con `sql_schema` e `column_mapping`;
- Esecuzione `ResponseProcessor`:

Elaborazione LLM e generazione database

- Streaming reasoning in tempo reale verso il frontend;
- Risposta finale strutturata con `sql_schema` e `column_mapping`;
- Esecuzione ResponseProcessor:
 - 1) Estrazione `sql_schema` ed estrazione tabelle;

Elaborazione LLM e generazione database

- Streaming reasoning in tempo reale verso il frontend;
- Risposta finale strutturata con `sql_schema` e `column_mapping`;
- Esecuzione ResponseProcessor:
 - 1) Estrazione `sql_schema` ed estrazione tabelle;
 - 2) Ordinamento topologico delle tabelle;

Elaborazione LLM e generazione database

- Streaming reasoning in tempo reale verso il frontend;
- Risposta finale strutturata con `sql_schema` e `column_mapping`;
- Esecuzione ResponseProcessor:
 - 1) Estrazione `sql_schema` ed estrazione tabelle;
 - 2) Ordinamento topologico delle tabelle;
 - 3) Creazione tabelle;

Elaborazione LLM e generazione database

- Streaming reasoning in tempo reale verso il frontend;
- Risposta finale strutturata con `sql_schema` e `column_mapping`;
- Esecuzione `ResponseProcessor`:
 - 1) Estrazione `sql_schema` ed estrazione tabelle;
 - 2) Ordinamento topologico delle tabelle;
 - 3) Creazione tabelle;
 - 3) Inserimento dati contenuti nei file mediante iteratori.

Elaborazione LLM e generazione database

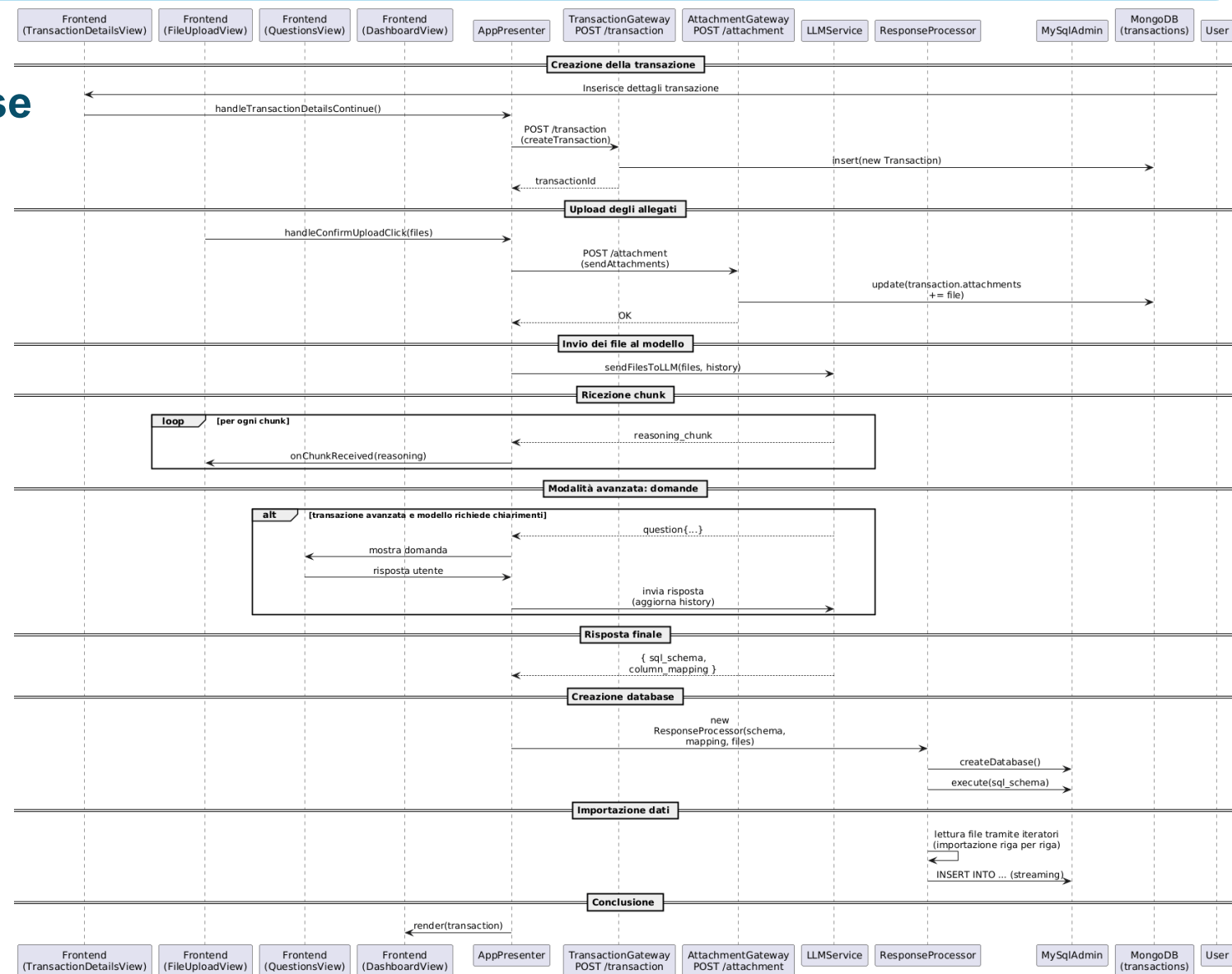
- Streaming reasoning in tempo reale verso il frontend;
- Risposta finale strutturata con `sql_schema` e `column_mapping`;
- Esecuzione ResponseProcessor:

1) Estrazione `sql_schema` ed estrazione tabelle;

2) Ordinamento topologico delle tabelle;

3) Creazione tabelle;

3) Inserimento dati contenuti nei file mediante iteratori.



Permessi sulle transazioni

Permessi sulle transazioni

- **Modello discezionale: proprietario, admin e gruppi autorizzati;**

Permessi sulle transazioni

- **Modello discezionale: proprietario, admin e gruppi autorizzati;**
- **Controllo su upload, advanced processing, dashboard, import ed eliminazioni.**

Vetrine transazioni

Vetrine transazioni

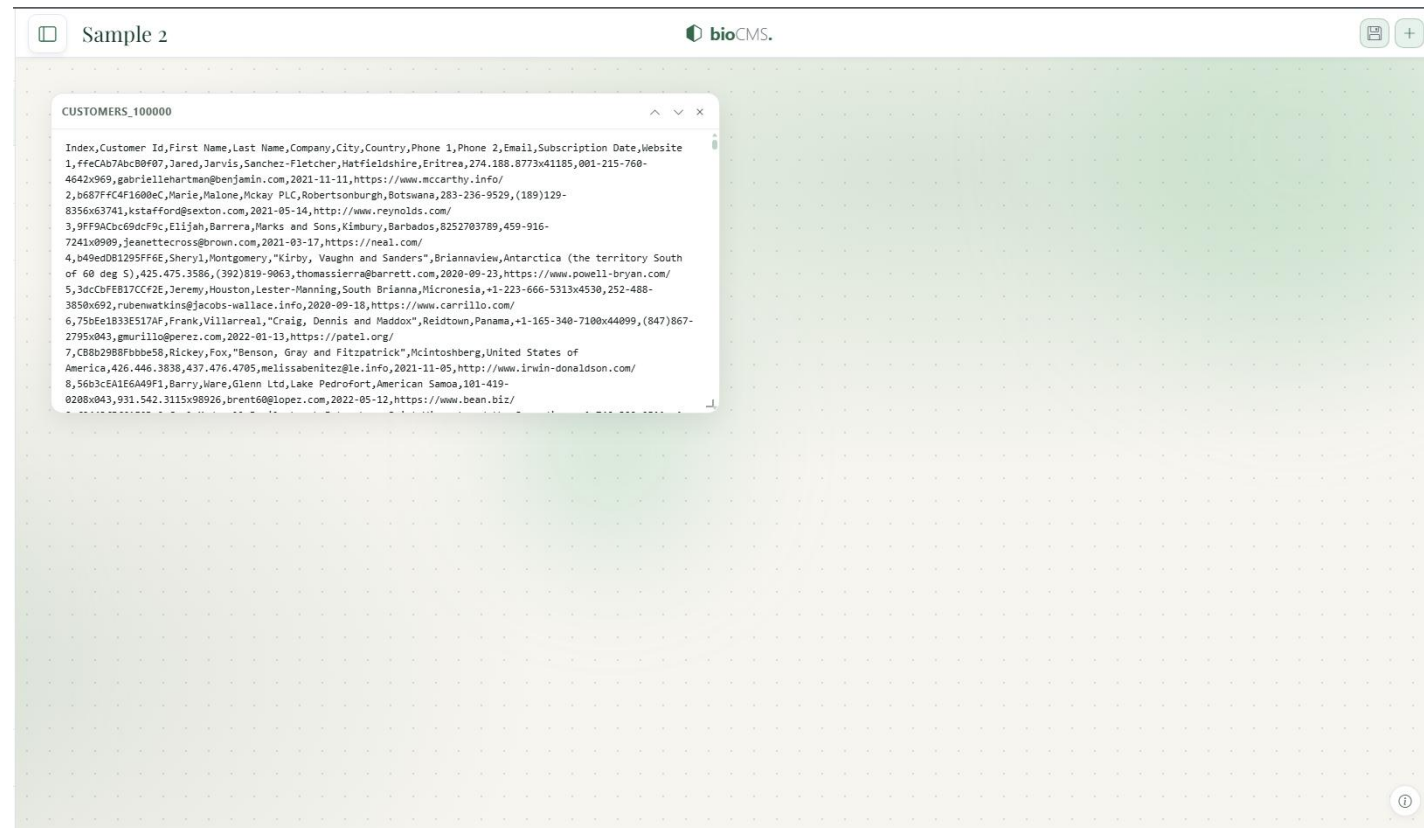
- Generazione automatica dei grafici basati sulle transazioni;

Vetrine transazioni

- Generazione automatica dei grafici basati sulle transazioni;
- Widget avanzati per la visualizzazione degli allegati alla transazione.

Vetrine transazioni

- Generazione automatica dei grafici basati sulle transazioni;
- Widget avanzati per la visualizzazione degli allegati alla transazione.



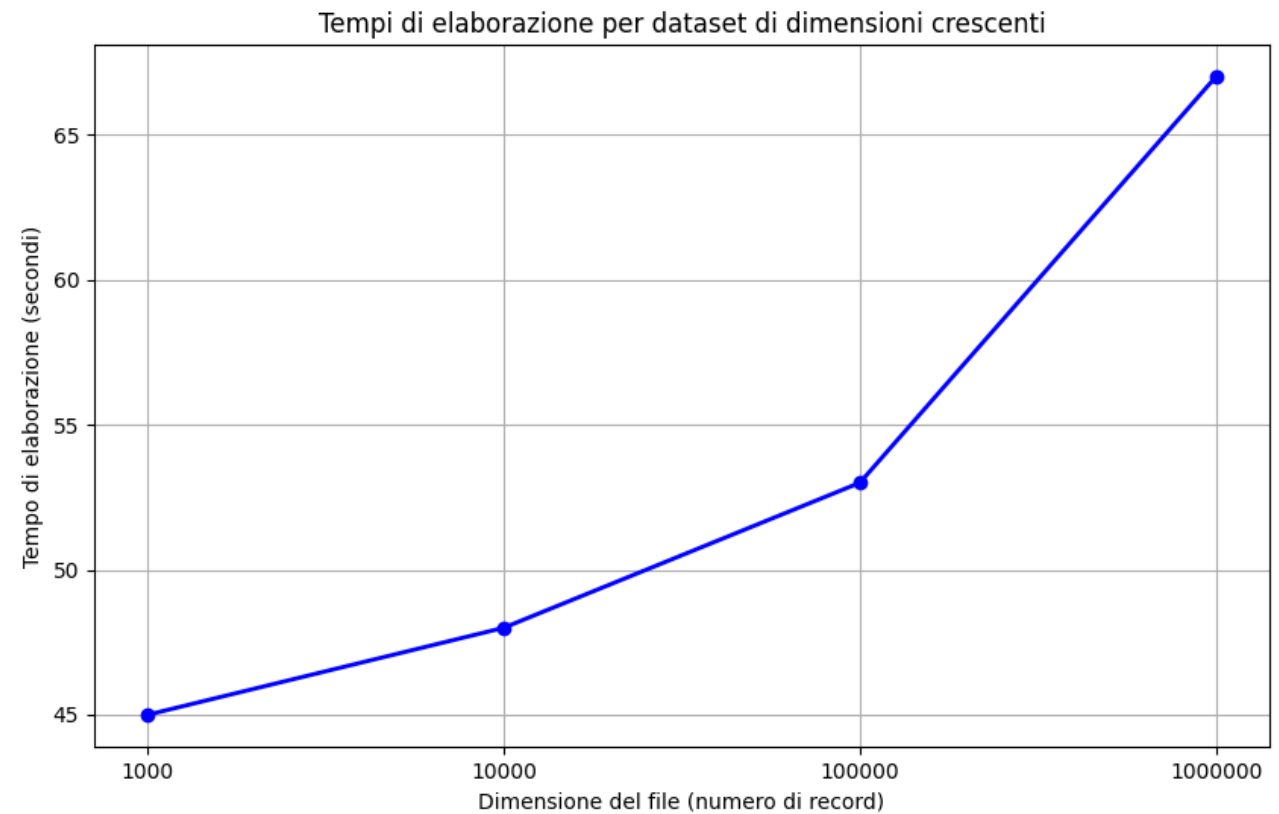
Prestazioni migliorate

Prestazioni migliorate

- Riduzione del tempo medio di elaborazione da 3 minuti a circa 1 minuto;

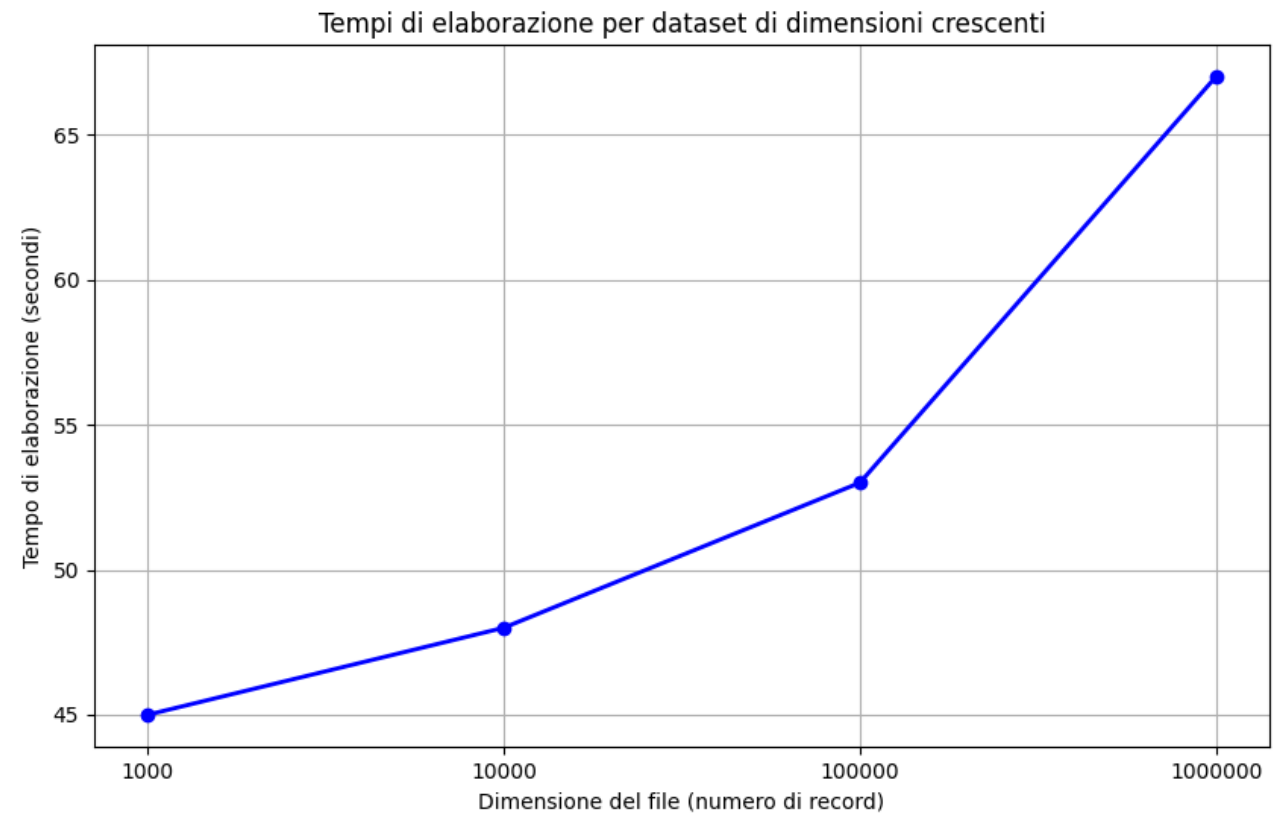
Prestazioni migliorate

- Riduzione del tempo medio di elaborazione da 3 minuti a circa 1 minuto;



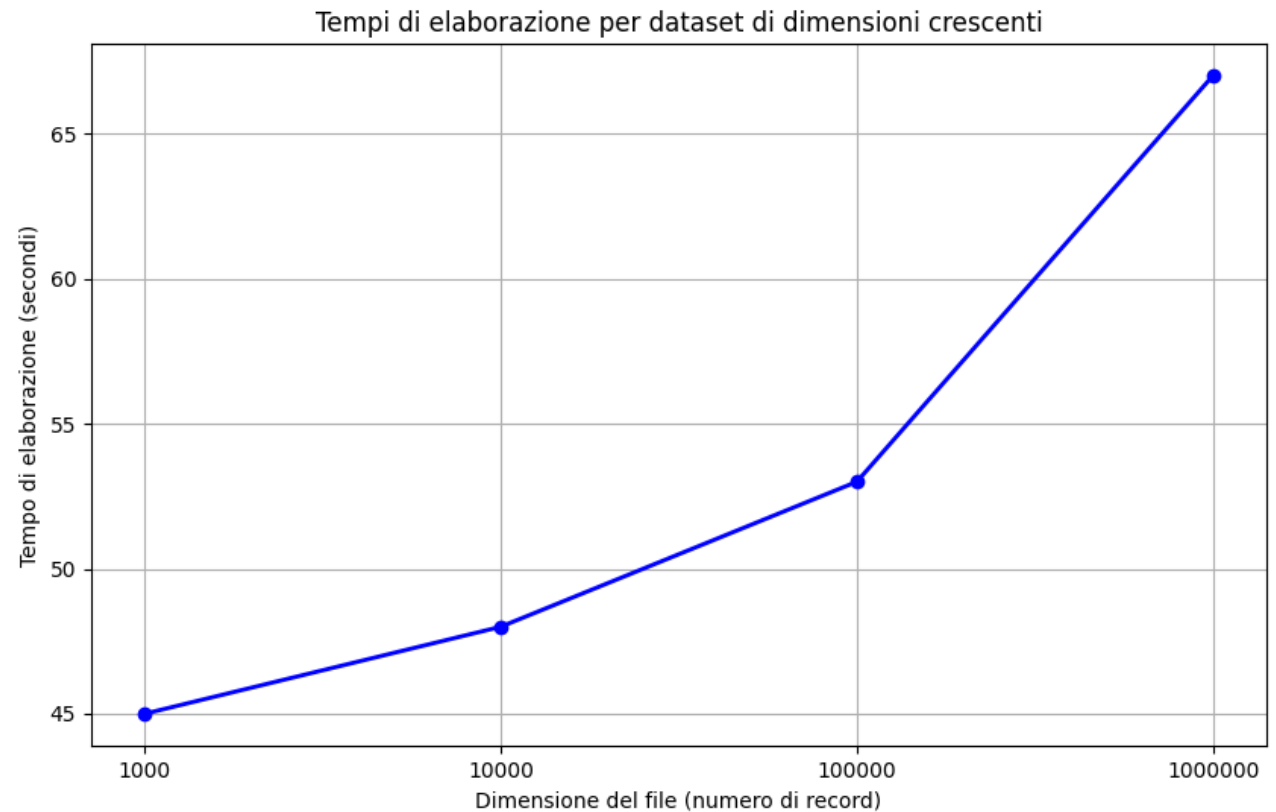
Prestazioni migliorate

- Riduzione del tempo medio di elaborazione da 3 minuti a circa 1 minuto;
- Miglior reattività grazie a:



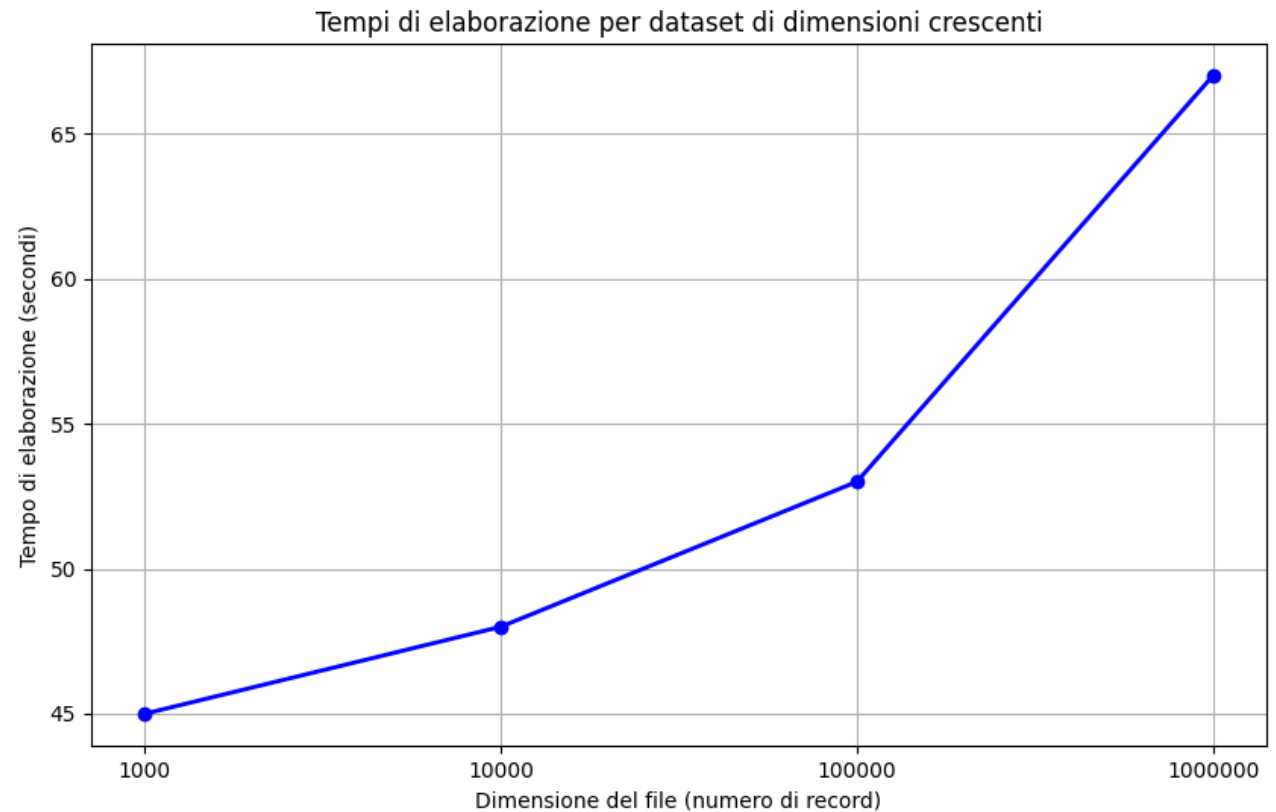
Prestazioni migliorate

- Riduzione del tempo medio di elaborazione da 3 minuti a circa 1 minuto;
- Miglior reattività grazie a:
 - 1) Caching delle risposte parziali;



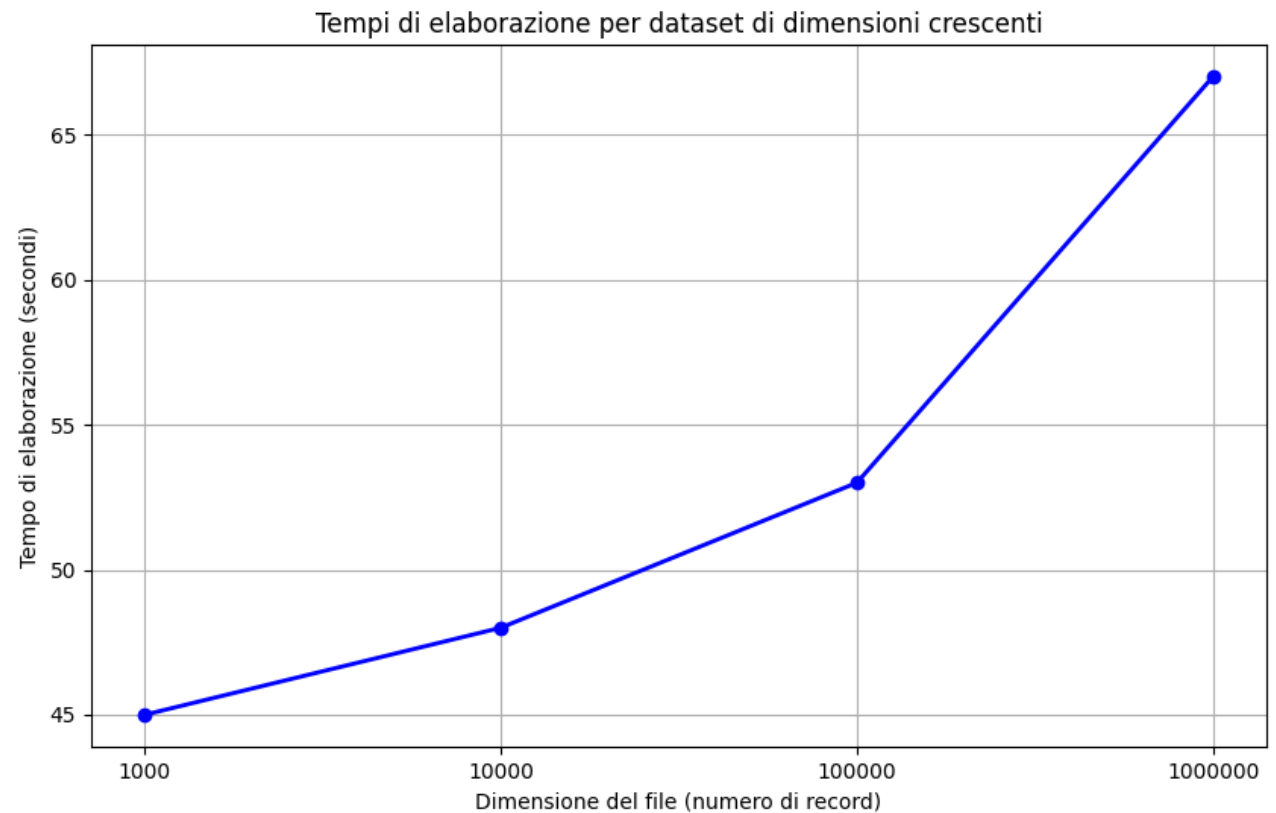
Prestazioni migliorate

- Riduzione del tempo medio di elaborazione da 3 minuti a circa 1 minuto;
- Miglior reattività grazie a:
 - 1) Caching delle risposte parziali;
 - 2) Ristrutturazione dell'analisi dell'output;



Prestazioni migliorate

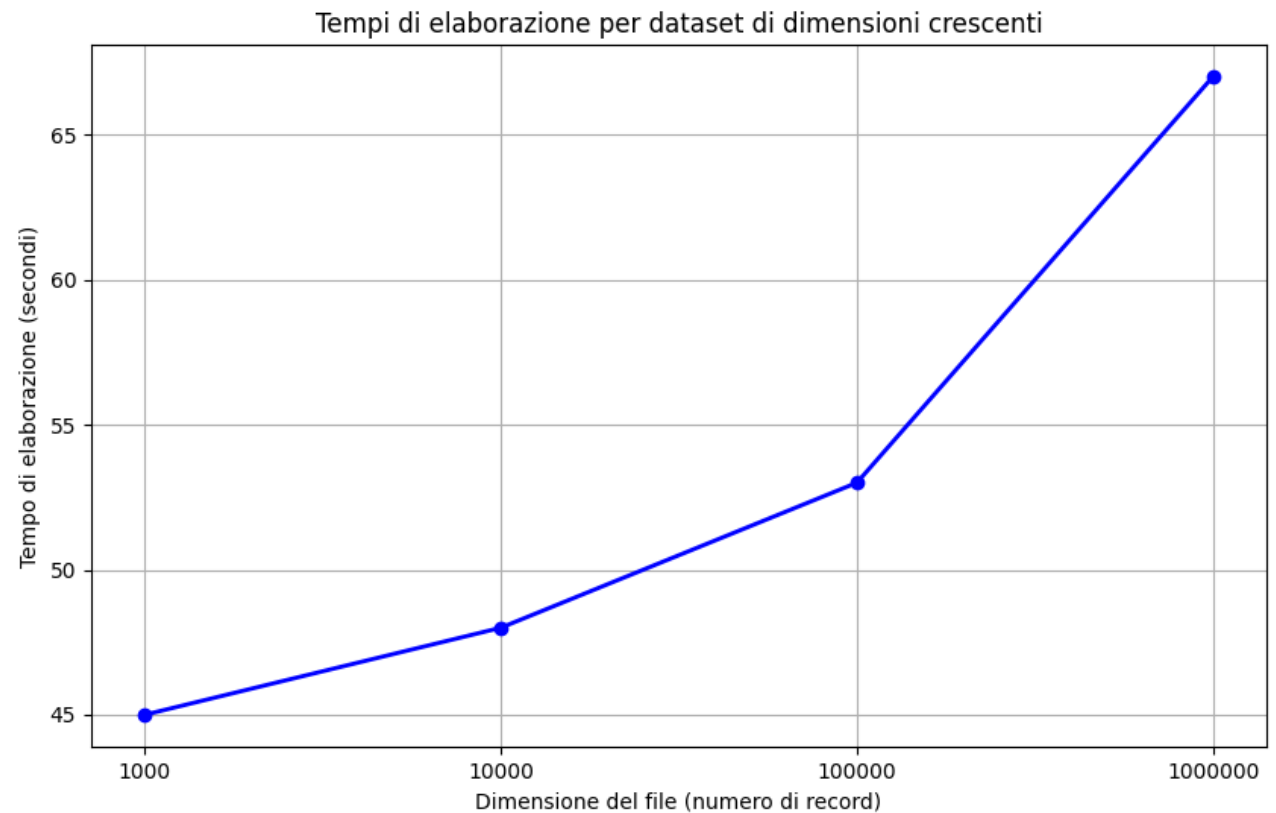
- Riduzione del tempo medio di elaborazione da 3 minuti a circa 1 minuto;
- Miglior reattività grazie a:
 - 1) Caching delle risposte parziali;
 - 2) Ristrutturazione dell'analisi dell'output;
 - 3) Gestione incrementale dei dati.



Prestazioni migliorate

- Riduzione del tempo medio di elaborazione da 3 minuti a circa 1 minuto;
- Miglior reattività grazie a:
 - 1) Caching delle risposte parziali;
 - 2) Ristrutturazione dell'analisi dell'output;
 - 3) Gestione incrementale dei dati.

Robustezza su dati eterogenei

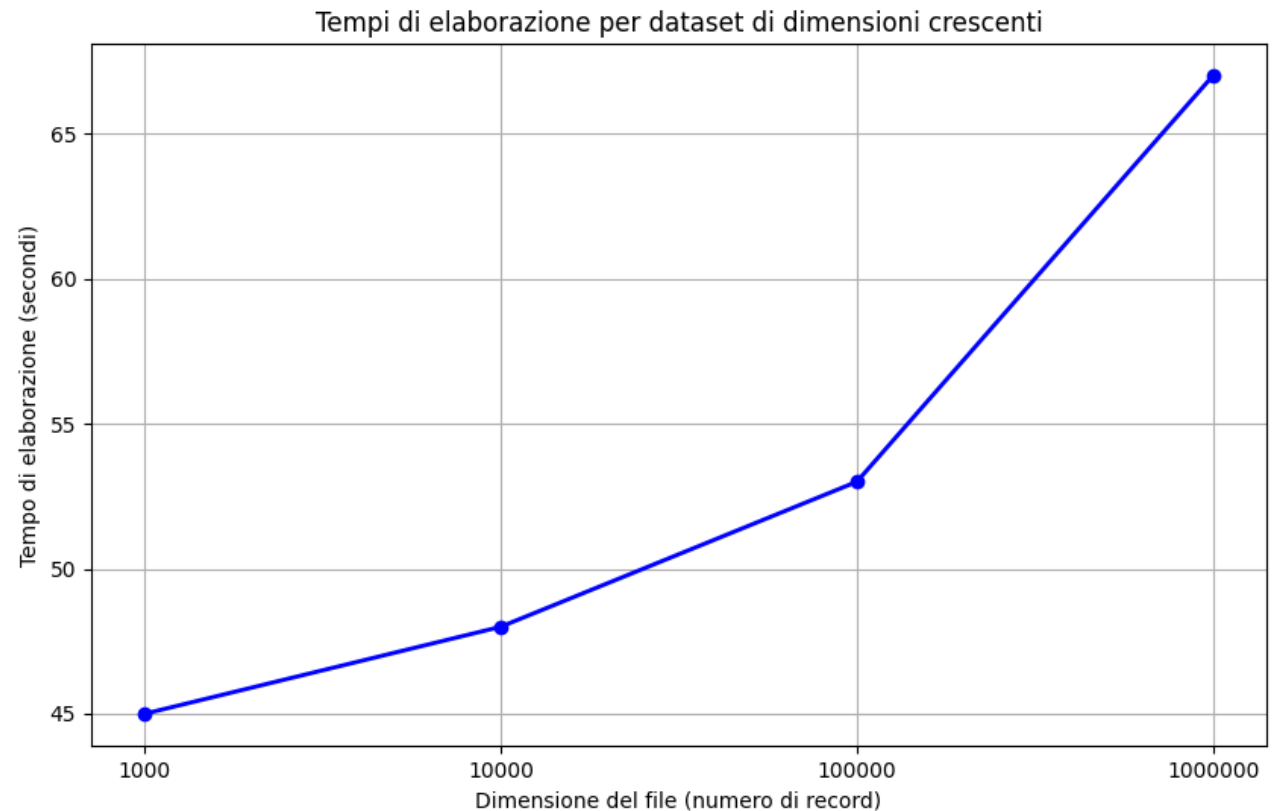


Prestazioni migliorate

- Riduzione del tempo medio di elaborazione da 3 minuti a circa 1 minuto;
- Miglior reattività grazie a:
 - 1) Caching delle risposte parziali;
 - 2) Ristrutturazione dell'analisi dell'output;
 - 3) Gestione incrementale dei dati.

Robustezza su dati eterogenei

- Test su dati con strutture diverse, distribuzioni non uniformi e qualità informativa variabile;

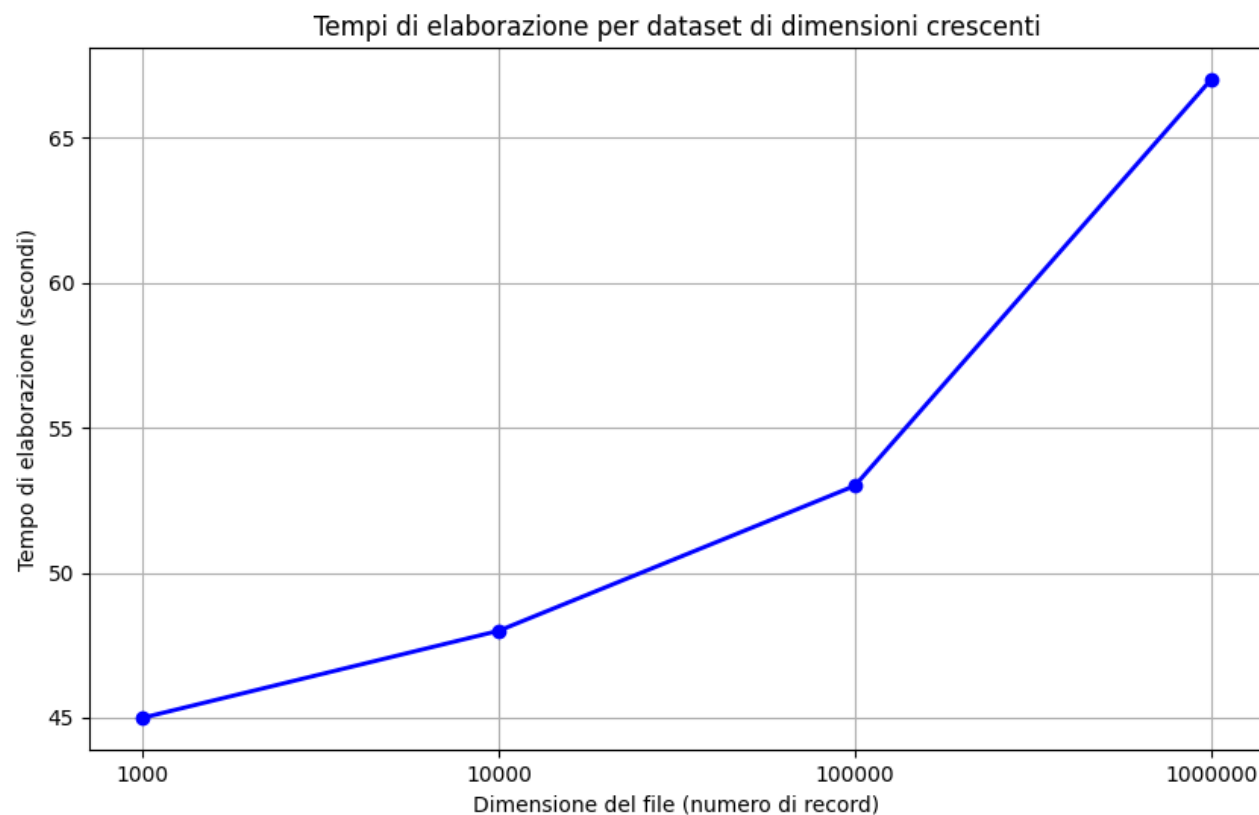


Prestazioni migliorate

- Riduzione del tempo medio di elaborazione da 3 minuti a circa 1 minuto;
- Miglior reattività grazie a:
 - 1) Caching delle risposte parziali;
 - 2) Ristrutturazione dell'analisi dell'output;
 - 3) Gestione incrementale dei dati.

Robustezza su dati eterogenei

- Test su dati con strutture diverse, distribuzioni non uniformi e qualità informativa variabile;
- Utilizzo di dataset mock con valori anomali, campi mancanti e schemi irregolari.



Risultato finale

Risultato finale

Sistema avanzato per l'analisi automatizzata di dataset eterogenei, con inferenza strutture dati e generazione schemi SQL coerenti.

Risultato finale

Sistema avanzato per l'analisi automatizzata di dataset eterogenei, con inferenza strutture dati e generazione schemi SQL coerenti.

Possibili sviluppi futuri

Risultato finale

Sistema avanzato per l'analisi automatizzata di dataset eterogenei, con inferenza strutture dati e generazione schemi SQL coerenti.

Possibili sviluppi futuri

- Completamento dashboard con nuovi viewer e widget interattivi;

Risultato finale

Sistema avanzato per l'analisi automatizzata di dataset eterogenei, con inferenza strutture dati e generazione schemi SQL coerenti.

Possibili sviluppi futuri

- Completamento dashboard con nuovi viewer e widget interattivi;
- Supporto a formati complessi (JSON annidati, XML, API esterne);

Risultato finale

Sistema avanzato per l'analisi automatizzata di dataset eterogenei, con inferenza strutture dati e generazione schemi SQL coerenti.

Possibili sviluppi futuri

- Completamento dashboard con nuovi viewer e widget interattivi;
- Supporto a formati complessi (JSON annidati, XML, API esterne);
- Esecuzione di algoritmi personalizzati sui dati, con salvataggio dell'output nella transazione;

Risultato finale

Sistema avanzato per l'analisi automatizzata di dataset eterogenei, con inferenza strutture dati e generazione schemi SQL coerenti.

Possibili sviluppi futuri

- Completamento dashboard con nuovi viewer e widget interattivi;
- Supporto a formati complessi (JSON annidati, XML, API esterne);
- Esecuzione di algoritmi personalizzati sui dati, con salvataggio dell'output nella transazione;
- Validazione automatica degli schemi, monitoraggio prestazioni, funzionalità collaborative;

Risultato finale

Sistema avanzato per l'analisi automatizzata di dataset eterogenei, con inferenza strutture dati e generazione schemi SQL coerenti.

Possibili sviluppi futuri

- Completamento dashboard con nuovi viewer e widget interattivi;
- Supporto a formati complessi (JSON annidati, XML, API esterne);
- Esecuzione di algoritmi personalizzati sui dati, con salvataggio dell'output nella transazione;
- Validazione automatica degli schemi, monitoraggio prestazioni, funzionalità collaborative;
- Esplorazione di tecniche di elaborazione distribuita per dataset più estesi.





Admin Setup

Create the administrator account to get started

FIRST NAME	LAST NAME
Fabio	Meloni

USERNAME



PASSWORD



CONFIRM PASSWORD



Create admin account



bioCMS.



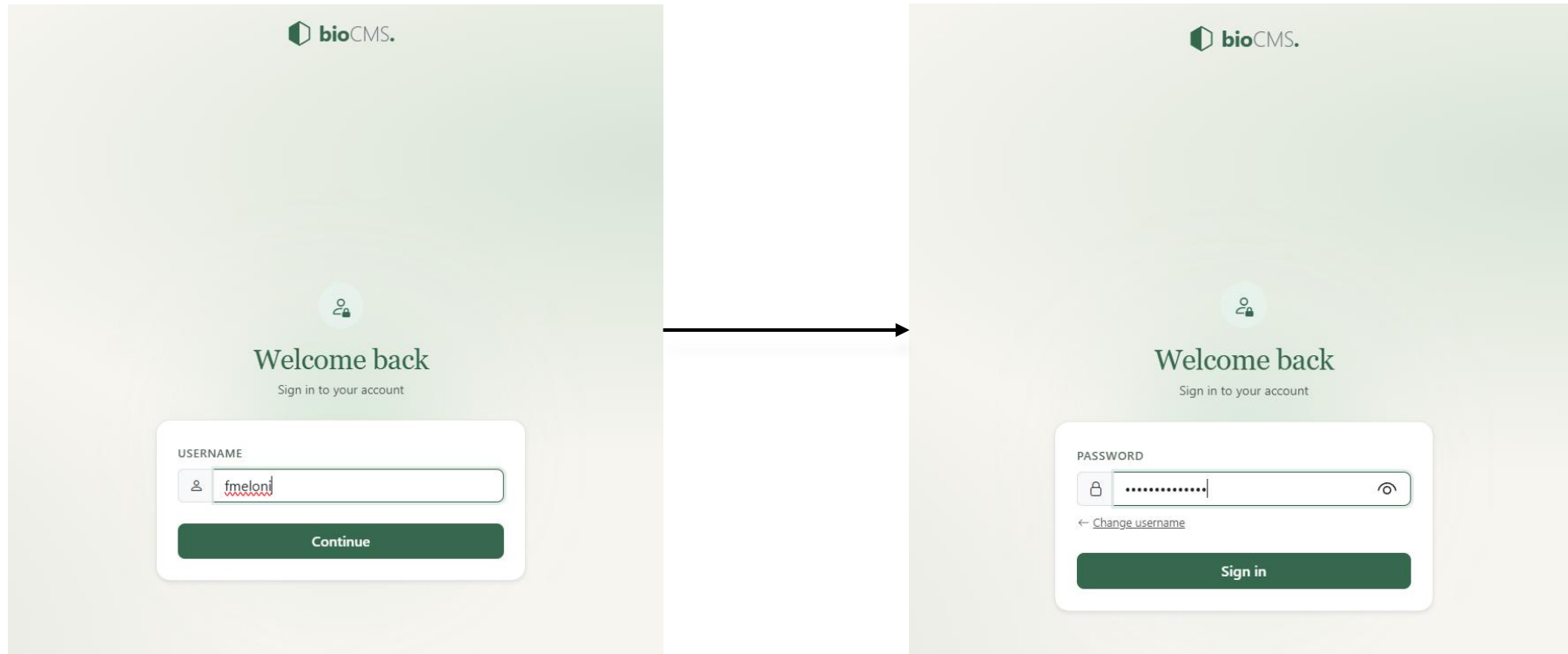
Welcome back

Sign in to your account

USERNAME



Continue



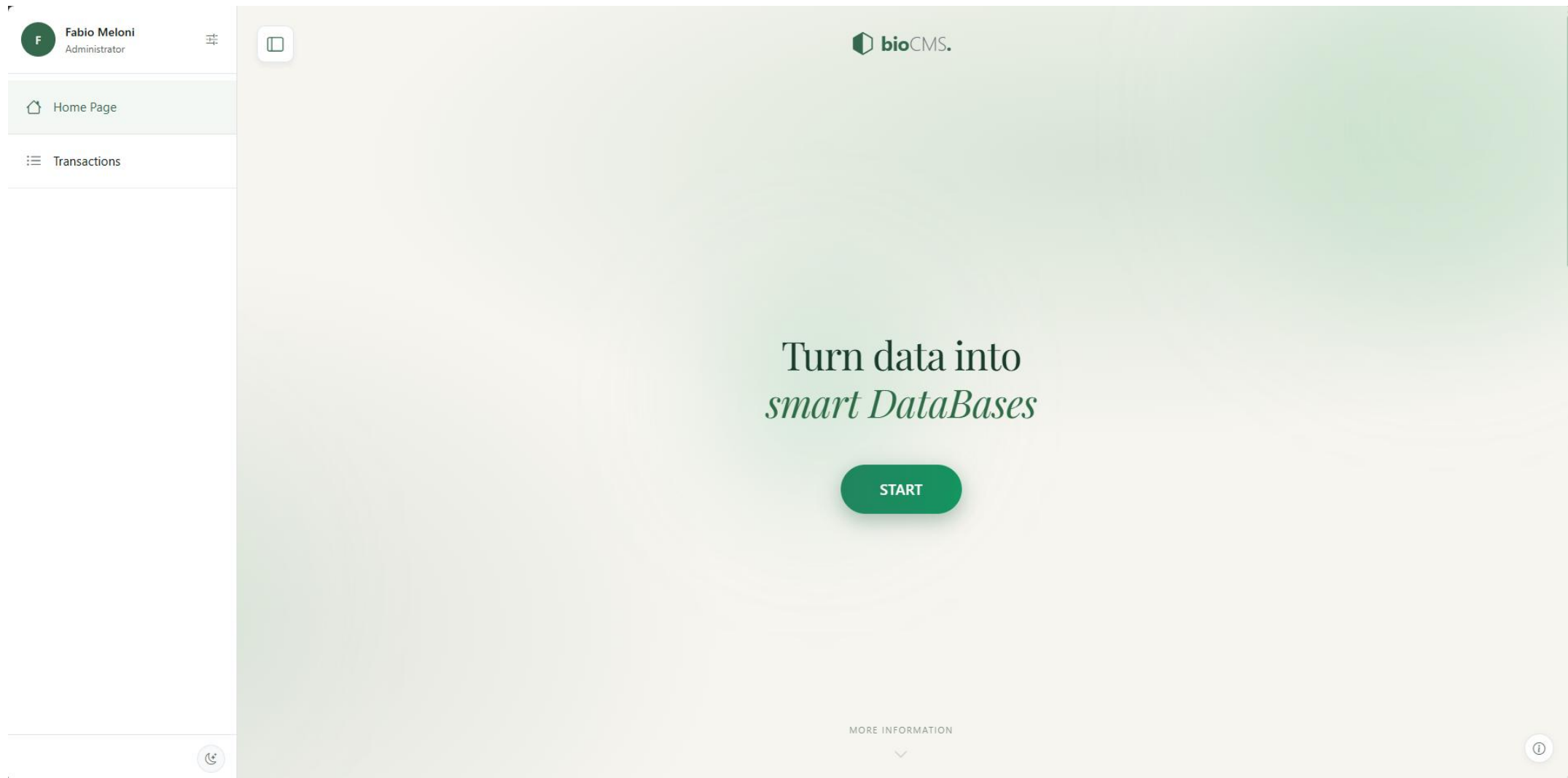
The diagram illustrates a two-step login process for the bioCMS application. It consists of two side-by-side screenshots of the login interface, connected by a horizontal arrow pointing from left to right.

Left Screenshot (Username Input):

- Header: bioCMS.
- Icon: A person icon in a circle.
- Text: "Welcome back" and "Sign in to your account".
- Form: A "USERNAME" input field containing the text "fmeloni".
- Button: A green "Continue" button.

Right Screenshot (Password Input):

- Header: bioCMS.
- Icon: A person icon in a circle.
- Text: "Welcome back" and "Sign in to your account".
- Form: A "PASSWORD" input field with masked characters (dots) and a toggle icon on the right.
- Link: A link labeled "← Change username" below the password field.
- Button: A green "Sign in" button.





Transaction Details

Expert mode ☐
Provide further information to the model by answering additional questions

Name
Sample 1

Description
Briefly describe this transaction

Categories
⊕ Add categories

Test Laboratorio 1 ✕

Continue

 bioCMS.

Transaction Details

Expert mode
Provide further information to the model by answering additional questions ☐

Name

Description

Categories

Test Laboratorio 1 ✕

Continue

Categories ✕

Search an existing category or create a new one

 **bioCMS.**

Transaction Details

Expert mode
Provide further information to the model by answering additional questions ☐

Name
Sample 1

Description
Briefly describe this transaction

Categories
+ Add categories

Test Laboratorio 1 ✕

Continue

Categories ✕
Search an existing category or create a new one

Search categories

+ Type a name to create a new category

Categories ✕
Search an existing category or create a new one

Test Laboratorio 1

No matching categories

+ Add "Test Laboratorio 1" as new category

 **Transaction Details**

Expert mode
Provide further information to the model by answering additional questions ☐

Name
Sample 1

Description
Briefly describe this transaction

Categories
+ Add categories
Test Laboratorio 1 ✕

Continue

Categories ✕
Search an existing category or create a new one

Search categories

+ Type a name to create a new category

Categories ✕
Search an existing category or create a new one

Test Laboratorio 1

No matching categories

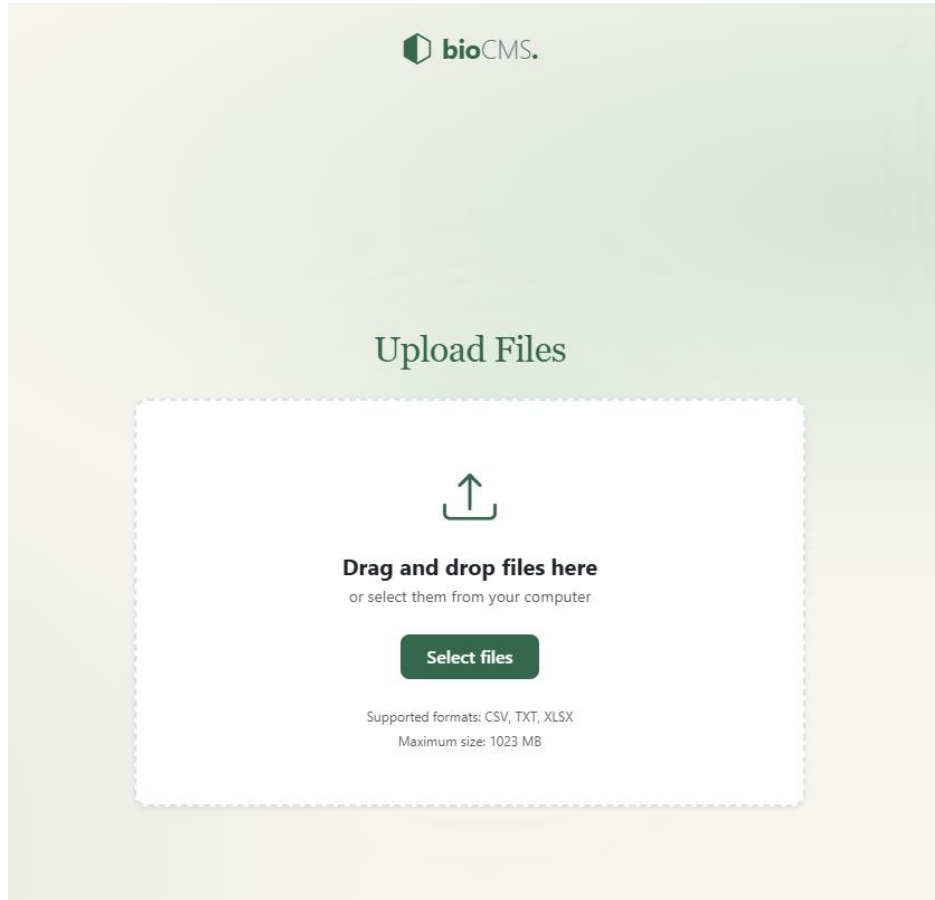
+ Add "Test Laboratorio 1" as new category

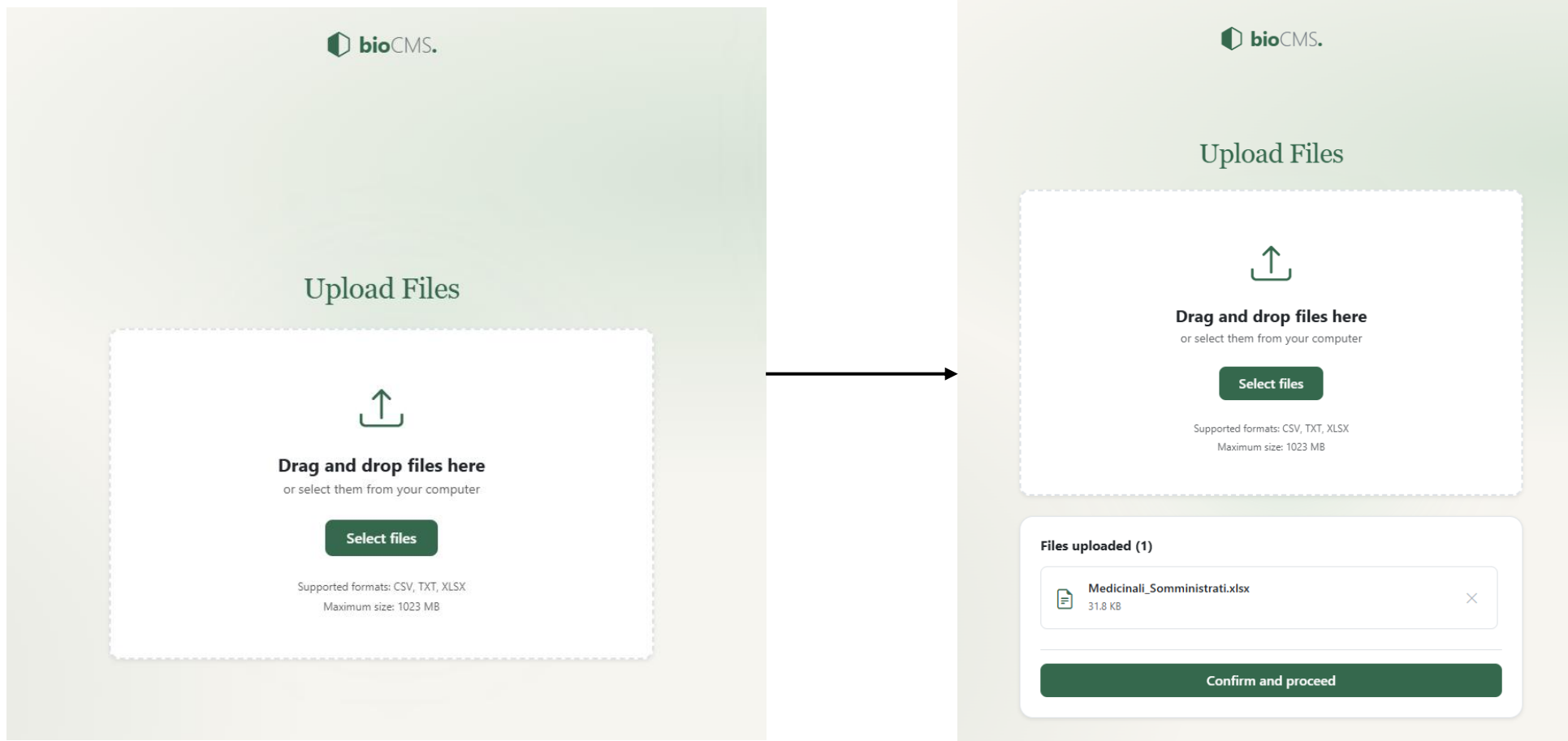
Categories ✕
Search an existing category or create a new one

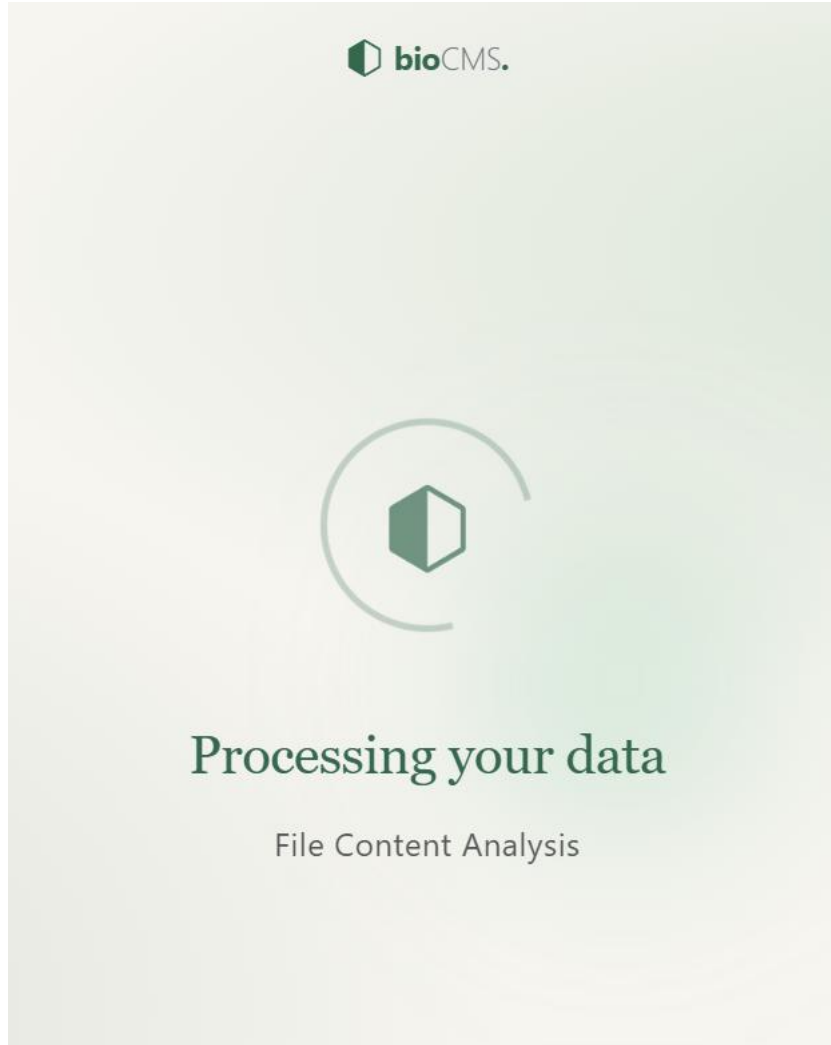
Search categories

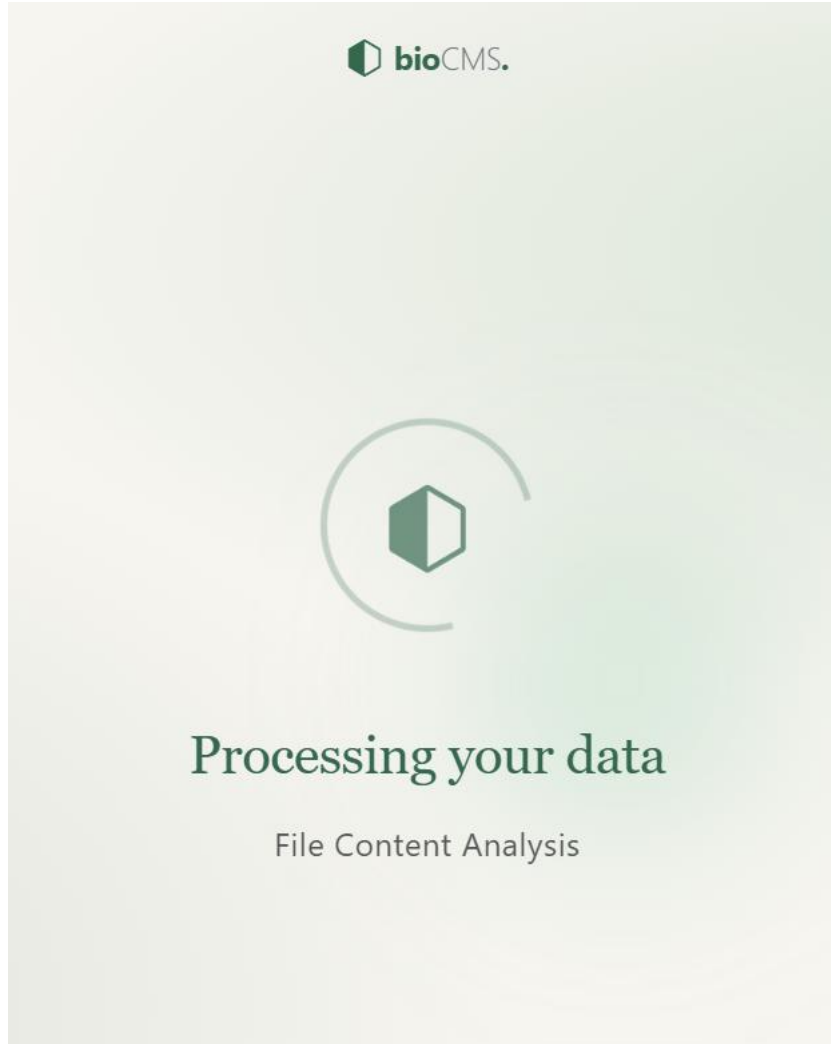
+ Type a name to create a new category

Test Laboratorio 1 ✕









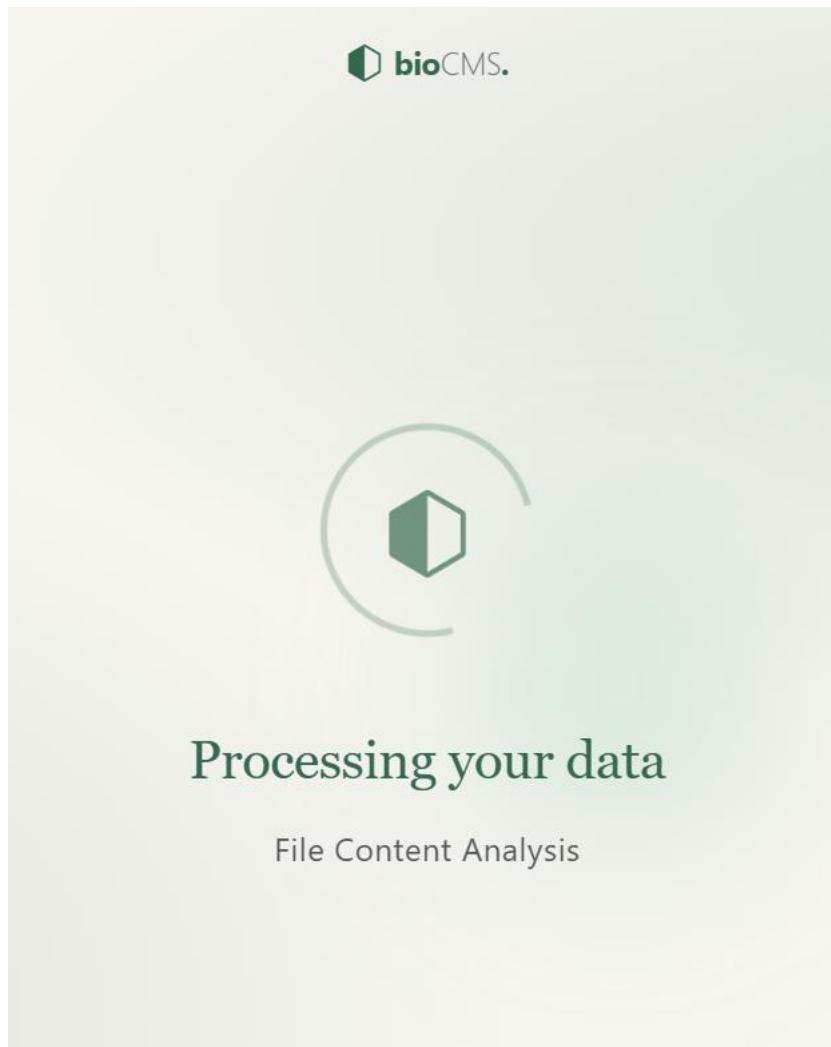
MODEL REASONING thinking >

To generate the SQL schema and column mapping for the provided CSV files, I will follow these steps:

1. **Infer Schema:** For each file that requires a schema, call `tool_files.infer_schema` to get the inferred schema.
2. **Sample Rows:** For each file that requires sample rows, call `tool_files.sample` to get a few sample rows.
3. **Analyze Sample Rows:** Use the sample rows to understand the data and relationships between tables.
4. **Generate SQL Schema:** Create the SQL schema based on the inferred schema and analysis of sample rows.
5. **Create Column Mapping:** Map each SQL column to its corresponding CSV column.

Let's start with the first file: "Reparti.csv".

i



MODEL REASONING thinking >

To generate the SQL schema and column mapping for the provided CSV files, I will follow these steps:

1. **Infer Schema:** For each file that requires a schema, call `tool_files.infer_schema` to get the inferred schema.
2. **Sample Rows:** For each file that requires sample rows, call `tool_files.sample` to get a few sample rows.
3. **Analyze Sample Rows:** Use the sample rows to understand the data and relationships between tables.
4. **Generate SQL Schema:** Create the SQL schema based on the inferred schema and analysis of sample rows.
5. **Create Column Mapping:** Map each SQL column to its corresponding CSV column.

Let's start with the first file: "Reparti.csv".

Based on the inferred schema for "Reparti.csv", I will create a table named `reparti` with columns `id_reparto`, `nome_reparto`, `piano`, and `responsabile`. The primary key will be `id_reparto`.

Next, I'll analyze the sample rows to understand if there are any relationships or additional constraints that need to be considered.

Sample rows:

1. `id_reparto: 1, nome_reparto: "Reparto A", piano: 2, responsabile: "Mario Rossi"`
2. `id_reparto: 2, nome_reparto: "Reparto B", piano: 3, responsabile: "Luigi Bianchi"`
3. `id_reparto: 3, nome_reparto: "Reparto C", piano: 1, responsabile: "Giuseppe Verdi"`

From the sample rows, it appears that each row represents a unique department with an ID, name, floor number and responsible person. ■



Processing your data

File Content Analysis

MODEL REASONING thinking >

To generate the SQL schema and column mapping for the provided CSV files, I will follow these steps:

1. **Infer Schema:** For each file that requires a schema, call `tool_files.infer_schema` to get the inferred schema.
2. **Sample Rows:** For each file that requires sample rows, call `tool_files.sample` to get a few sample rows.
3. **Analyze Sample Rows:** Use the sample rows to understand the data and relationships between tables.
4. **Generate SQL Schema:** Create the SQL schema based on the inferred schema and analysis of sample rows.
5. **Create Column Mapping:** Map each SQL column to its corresponding CSV column.

Let's start with the first file: "Reparti.csv".

Based on the inferred schema for "Reparti.csv", I will create a table named `reparti` with columns `id_reparto`, `nome_reparto`, `piano`, and `responsabile`. The primary key will be `id_reparto`. Next, I'll analyze the sample rows to understand if there are any relationships or additional constraints that need to be considered.

Sample rows:

1. `id_reparto: 1, nome_reparto: "Reparto A", piano: 2, responsabile: "Mario Rossi"`
2. `id_reparto: 2, nome_reparto: "Reparto B", piano: 3, responsabile: "Luigi Bianchi"`
3. `id_reparto: 3, nome_reparto: "Reparto C", piano: 1, responsabile: "Giuseppe Verdi"`

From the sample rows, it appears that each row represents a unique department with an ID, name, floor number and responsible person. ■

From the sample rows, it appears that each row represents a unique department with an ID, name, floor number, and responsible person.

Based on the sample rows, I will finalize the SQL schema for the `reparti` table and create a column mapping.

SQL Schema:

```
CREATE TABLE reparti (  
  id_reparto INT NOT NULL COMMENT 'U  
  nome_reparto VARCHAR(255) NOT NULL  
  piano INT NOT NULL COMMENT 'Floor  
  responsabile VARCHAR(255) NOT NULL  
  PRIMARY KEY (id_reparto)  
) COMMENT='Table containing informati
```

Column Mapping:

```
{  
  "reparti.id_reparto": "Reparti.csv.i  
  "reparti.nome_reparto": "Reparti.csv  
  "reparti.piano": "Reparti.csv.piano"  
  "reparti.responsabile": "Reparti.csv  
}
```

This schema and mapping ensure that each column in the SQL table corresponds to a specific column in the CSV file, and the primary key is correctly identified.

MODEL REASONING thinking >

To generate the SQL schema and column mapping for the provided CSV files, I will follow these steps:

1. **Infer Schema:** For each file that requires a schema, call `tool_files.infer_schema` to get the inferred schema.
2. **Sample Rows:** For each file that requires sample rows, call `tool_files.sample` to get a few sample rows.
3. **Analyze Sample Rows:** Use the sample rows to understand the data and relationships between tables.
4. **Generate SQL Schema:** Create the SQL schema based on the inferred schema and analysis of sample rows.
5. **Create Column Mapping:** Map each SQL column to its corresponding CSV column.

Let's start with the first file: "Reparti.csv".

Based on the inferred schema for "Reparti.csv", I will create a table named `reparti` with columns `id_reparto`, `nome_reparto`, `piano`, and `responsabile`. The primary key will be `id_reparto`. Next, I'll analyze the sample rows to understand if there are any relationships or additional constraints that need to be considered.

Sample rows:

1. `id_reparto: 1, nome_reparto: "Reparto A", piano: 2, responsabile: "Mario Rossi"`
2. `id_reparto: 2, nome_reparto: "Reparto B", piano: 3, responsabile: "Luigi Bianchi"`
3. `id_reparto: 3, nome_reparto: "Reparto C", piano: 1, responsabile: "Giuseppe Verdi"`

From the sample rows, it appears that each row represents a unique department with an ID, name, floor number and responsible person. ■

From the sample rows, it appears that each row represents a unique department with an ID, name, floor number, and responsible person.

Based on the sample rows, I will finalize the SQL schema for the `reparti` table and create a column mapping.

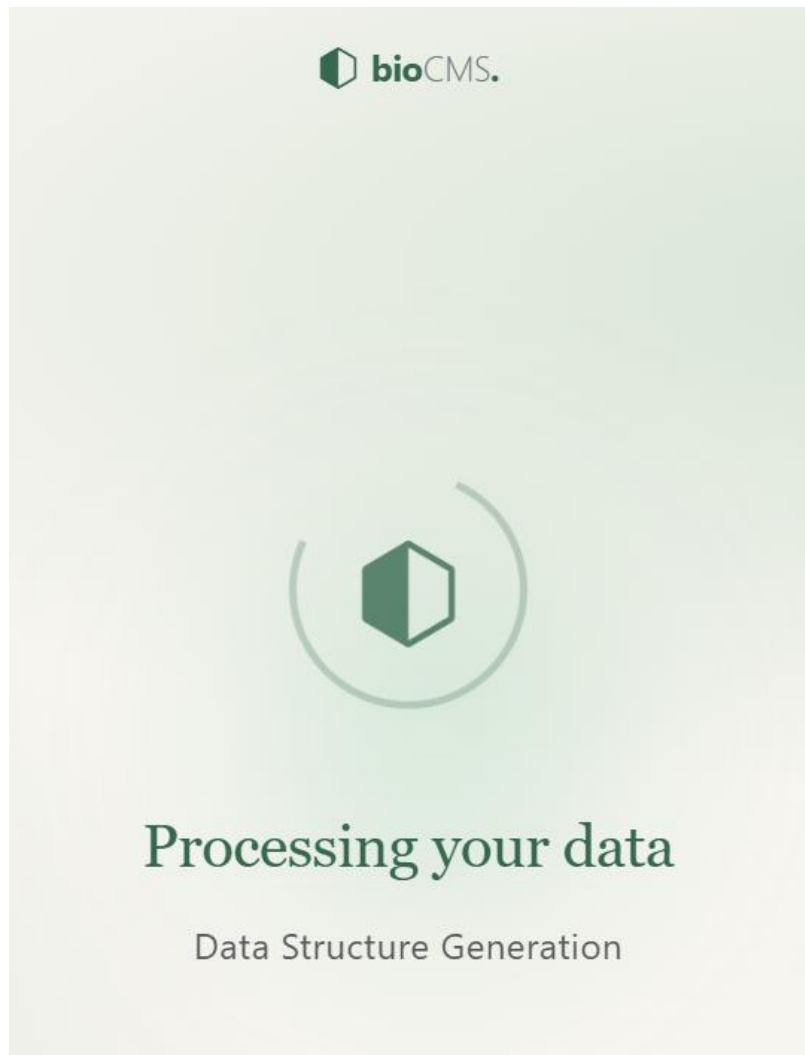
SQL Schema:

```
CREATE TABLE reparti (  
  id_reparto INT NOT NULL COMMENT 'U  
  nome_reparto VARCHAR(255) NOT NULL  
  piano INT NOT NULL COMMENT 'Floor  
  responsabile VARCHAR(255) NOT NULL  
  PRIMARY KEY (id_reparto)  
) COMMENT='Table containing informati
```

Column Mapping:

```
{  
  "reparti.id_reparto": "Reparti.csv.i  
  "reparti.nome_reparto": "Reparti.csv  
  "reparti.piano": "Reparti.csv.piano"  
  "reparti.responsabile": "Reparti.csv  
}
```

This schema and mapping ensure that each column in the SQL table corresponds to a specific column in the CSV file, and the primary key is correctly identified.



MODEL REASONING thinking >

To generate the SQL schema and column mapping for the provided CSV files, I will follow these steps:

1. **Infer Schema:** For each file that requires a schema, call `tool_files.infer_schema` to get the inferred schema.
2. **Sample Rows:** For each file that requires sample rows, call `tool_files.sample` to get a few sample rows.
3. **Analyze Sample Rows:** Use the sample rows to understand the data and relationships between tables.
4. **Generate SQL Schema:** Create the SQL schema based on the inferred schema and analysis of sample rows.
5. **Create Column Mapping:** Map each SQL column to its corresponding CSV column.

Let's start with the first file: "Reparti.csv".

Based on the inferred schema for "Reparti.csv", I will create a table named `reparti` with columns `id_reparto`, `nome_reparto`, `piano`, and `responsabile`. The primary key will be `id_reparto`. Next, I'll analyze the sample rows to understand if there are any relationships or additional constraints that need to be considered.

Sample rows:

1. `id_reparto: 1, nome_reparto: "Reparto A", piano: 2, responsabile: "Mario Rossi"`
2. `id_reparto: 2, nome_reparto: "Reparto B", piano: 3, responsabile: "Luigi Bianchi"`
3. `id_reparto: 3, nome_reparto: "Reparto C", piano: 1, responsabile: "Giuseppe Verdi"`

From the sample rows, it appears that each row represents a unique department with an ID, name, floor number and responsible person. ■

From the sample rows, it appears that each row represents a unique department with an ID, name, floor number, and responsible person.

Based on the sample rows, I will finalize the SQL schema for the `reparti` table and create a column mapping.

SQL Schema:

```
CREATE TABLE reparti (  
  id_reparto INT NOT NULL COMMENT 'U  
  nome_reparto VARCHAR(255) NOT NULL  
  piano INT NOT NULL COMMENT 'Floor  
  responsabile VARCHAR(255) NOT NULL  
  PRIMARY KEY (id_reparto)  
) COMMENT='Table containing informati
```

Column Mapping:

```
{  
  "reparti.id_reparto": "Reparti.csv.i  
  "reparti.nome_reparto": "Reparti.csv  
  "reparti.piano": "Reparti.csv.piano"  
  "reparti.responsabile": "Reparti.csv  
}
```

This schema and mapping ensure that each column in the SQL table corresponds to a specific column in the CSV file, and the primary key is correctly identified.

MODEL REASONING thinking >

To generate the SQL schema and column mapping for the provided CSV files, I will follow these steps:

1. **Infer Schema:** For each file that requires a schema, call `tool_files.infer_schema` to get the inferred schema.
2. **Sample Rows:** For each file that requires sample rows, call `tool_files.sample` to get a few sample rows.
3. **Analyze Sample Rows:** Use the sample rows to understand the data and relationships between tables.
4. **Generate SQL Schema:** Create the SQL schema based on the inferred schema and analysis of sample rows.
5. **Create Column Mapping:** Map each SQL column to its corresponding CSV column.

Let's start with the first file: "Reparti.csv".

Based on the inferred schema for "Reparti.csv", I will create a table named `reparti` with columns `id_reparto`, `nome_reparto`, `piano`, and `responsabile`. The primary key will be `id_reparto`. Next, I'll analyze the sample rows to understand if there are any relationships or additional constraints that need to be considered.

Sample rows:

1. `id_reparto: 1, nome_reparto: "Reparto A", piano: 2, responsabile: "Mario Rossi"`
2. `id_reparto: 2, nome_reparto: "Reparto B", piano: 3, responsabile: "Luigi Bianchi"`
3. `id_reparto: 3, nome_reparto: "Reparto C", piano: 1, responsabile: "Giuseppe Verdi"`

From the sample rows, it appears that each row represents a unique department with an ID, name, floor number and responsible person. ■

From the sample rows, it appears that each row represents a unique department with an ID, name, floor number, and responsible person.

Based on the sample rows, I will finalize the SQL schema for the `reparti` table and create a column mapping.

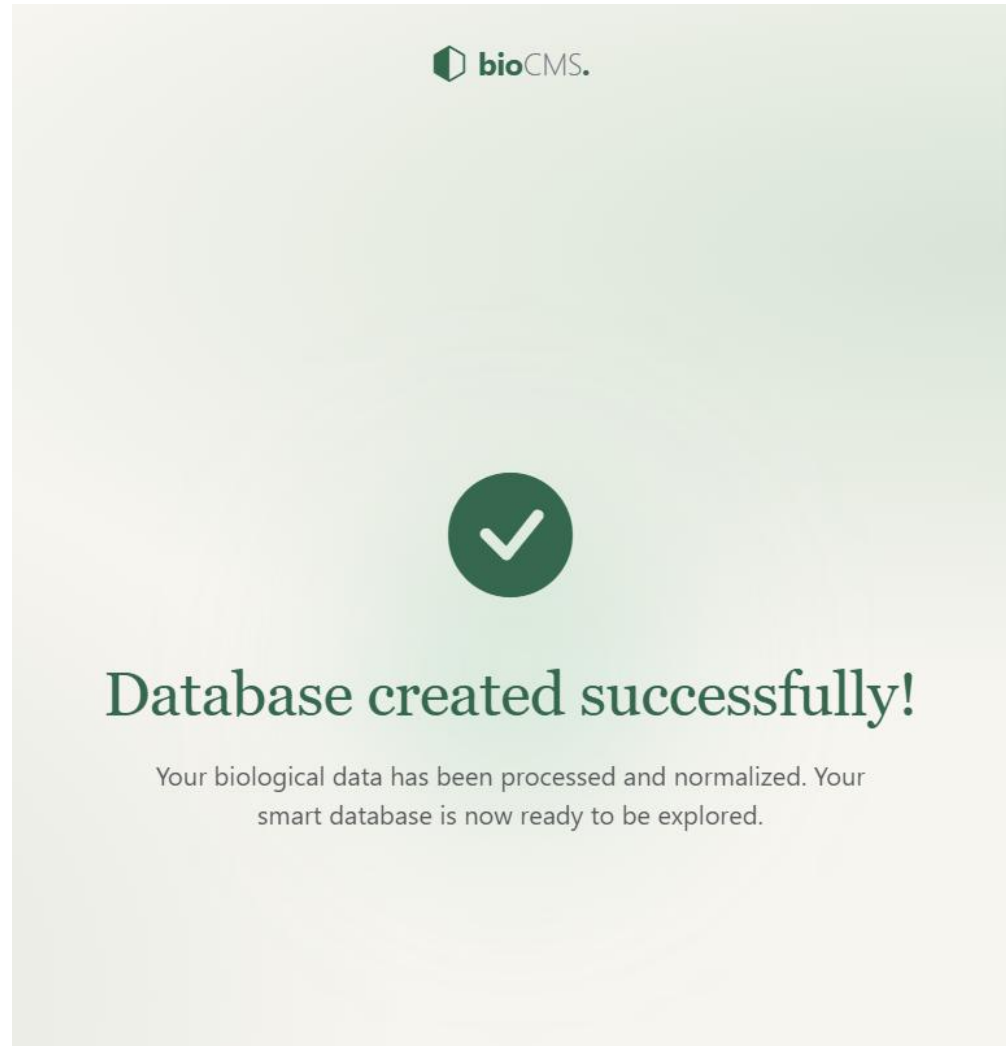
SQL Schema:

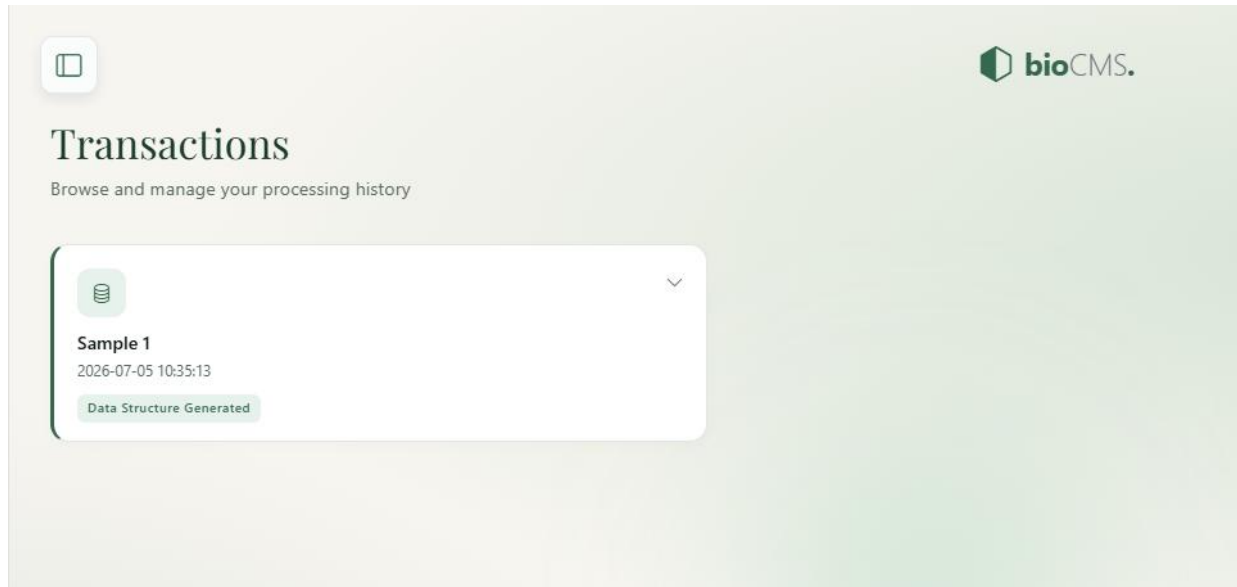
```
CREATE TABLE reparti (  
  id_reparto INT NOT NULL COMMENT 'U  
  nome_reparto VARCHAR(255) NOT NULL  
  piano INT NOT NULL COMMENT 'Floor  
  responsabile VARCHAR(255) NOT NULL  
  PRIMARY KEY (id_reparto)  
) COMMENT='Table containing informati
```

Column Mapping:

```
{  
  "reparti.id_reparto": "Reparti.csv.i  
  "reparti.nome_reparto": "Reparti.csv  
  "reparti.piano": "Reparti.csv.piano"  
  "reparti.responsabile": "Reparti.csv  
}
```

This schema and mapping ensure that each column in the SQL table corresponds to a specific column in the CSV file, and the primary key is correctly identified.





Sample 1 READ ONLY

TRAINING PROMPT

System prompt injected at the start of the transaction.

ATTACHMENTS



Medicinali_Somministrati.xlsx

Click to preview



BIOCMS

SQL Schema

```
CREATE TABLE reparti (  
  id_reparto INT NOT NULL COMMENT 'Unique identifier for each department',  
  nome_reparto VARCHAR(255) NOT NULL COMMENT 'Name of the department',  
  piano INT NOT NULL COMMENT 'Floor number where the department is located',  
  responsabile VARCHAR(255) NOT NULL COMMENT 'Responsible person for the depart',  
  PRIMARY KEY (id_reparto)  
) COMMENT='Table containing information about hospital departments';
```

Column Mapping

```
reparti.id_reparto → Reparti.csv.id_reparto  
reparti.nome_reparto → Reparti.csv.nome_reparto  
reparti.piano → Reparti.csv.piano  
reparti.responsabile → Reparti.csv.responsabile
```

